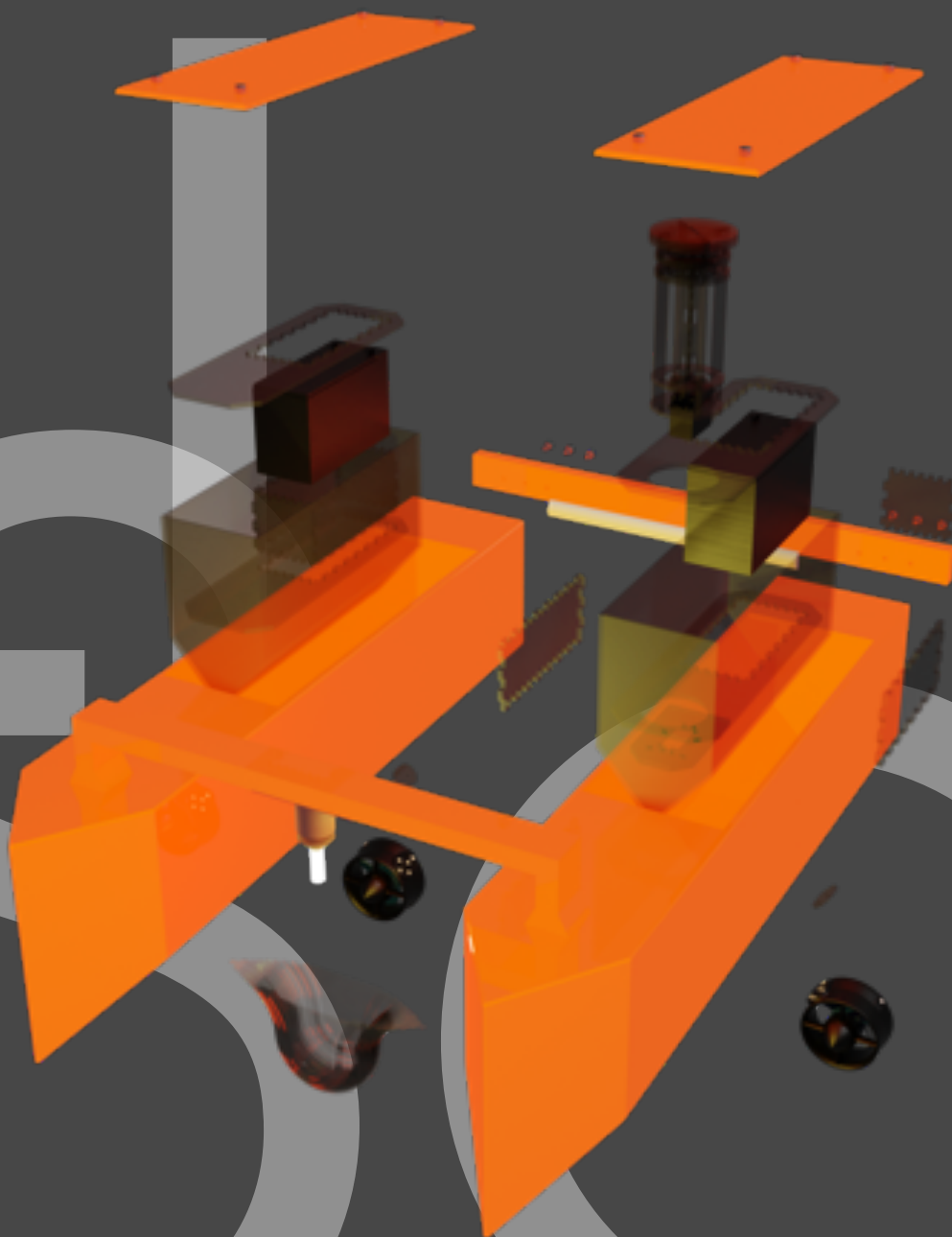




Semester Project 3

Report

Almothana Al Moslem, Hritik Roy Chowdhury,
Yiheng Liu, Zuzanna Parnicka, Vojtech Ilčík
31.12.2023

SDU 

Abstract

LiSa boat is a an autonomous life saver boat, that was developed with the intention to save lives of drowning people. Thanks to its autonomous navigation it can act as a lifeguard in less serious cases or act as a support during the serious cases.

Report follows a design process from the idea generation until the testing phase. In the beginning, there is justification of the competitiveness by comparing to other available solution and feedback from lifeguards. Body is divided into the three main parts that are describing particular steps in the hardware and software development.

Acknowledgements

We extend our sincere gratitude to Kasper Paasch for his invaluable guidance and support throughout this autonomous rescue boat project. Special thanks to SDU Sønderborg for providing resources. We appreciate the dedication of our team members and the input from user testing/surveying participants. Thanks also to friends and family for their support and encouragement. This project's success is a result of the collective efforts of everyone involved. A special mention goes to Jerome Jouffroy, who initially assisted us and played a key role in shaping the project's early stages.

Project Responsibilities and Signature

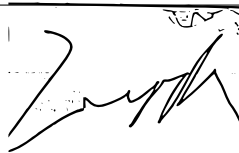
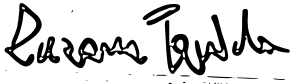
Name	Project Responsibilities	Signature
Almothana Al Moslem	Electrical and Electronics	
Hritik Roy Chowdhury	Electronics and Programming	
Yiheng Liu	Mechanical, prototype programming (RF module)	
Zuzanna Parnicka	Mechanical and Electrical (Voltage Regulator), prototype programming (RF module)	
Vojtech Ilčík	Mechanical and Electrical, Management	

Table of Contents

1	Introduction	4
2	Problem formulation	5
2.1	Market research	5
2.2	User research	6
2.3	User requirements	6
2.4	Product requirements	6
2.4.1	Need to have requirements	6
2.4.2	Nice to have requirements	7
2.5	Risk assesment	7
2.5.1	Standards to be satisfied	8
2.5.2	Risk Management	8
3	Theoretical analysis	11
3.1	Component selection	11
3.1.1	What to consider before choosing GPS module	11
3.1.2	Considered options and their specifications	11
4	Project Management	13
4.1	Stage gate model implementation	13
4.2	Task management	13
5	Mechanical design	14
5.1	Boat's components housing	14
5.2	Calibration Tower	15
5.2.1	Calibration needs	15
5.2.2	Instruments installation and accessibility	16
5.2.3	Design process	18
5.3	LED lamp	18
5.3.1	Illumination needs	18
5.3.2	LED design	19
5.4	Handlebar	19
5.4.1	Handlebar requirements	20
5.4.2	Handlebar manufacturing	20
5.5	Bracelet	20
6	Electrical design	22
6.1	Bracelet PCB	22
6.2	Power Supply	23
6.2.1	Power supply design	23
6.2.2	Power supply test	23
6.3	PCB Design	24
6.4	Boat PCB	24
6.5	Transducer RX Circuit	25

6.5.1	MFB Band Pass Filter:	25
6.5.2	Inverting amplifier	28
6.5.3	Signal processing	28
6.5.4	Peak-Detector Circuit	29
6.6	Transducer TX circuit	30
6.6.1	MFB Band Pass Filter:	31
6.6.2	Common Emitter Amplifier:	31
6.7	Main PCB	32
7	Programming and Algorithms	33
7.1	GPS Module	33
7.1.1	Introduction to GPS	33
7.1.2	Parsing of NMEA Sentences	33
7.1.3	UART Setup	35
7.2	RF Module	35
7.2.1	Introduction to RF	35
7.2.2	GPS Precision for RF Communication	36
7.2.3	Transmission: Cycle and Typecasting	36
7.2.4	Transmission: Extracting Digits	37
7.2.5	Receiving: Rebuilding Coordinates	38
7.2.6	Transmitting: Activating Starting Protocols	39
7.3	Digital Compass	40
7.3.1	Introduction to Digital Compass	40
7.3.2	Calibration and Heading Calculation	41
7.4	Magnetometer Tilt Compensation	46
7.4.1	Problems with Tilt	46
7.4.2	Tilt Compensation Equations	46
7.4.3	Getting Roll and Pitch Angles for Tilt Compensation	47
7.4.4	Sensor Fusion with Kalman Filtering	49
7.4.5	Tilt Compensation Results	50
7.5	Navigation	53
7.5.1	Motor Control, Navigation Setup and Algorithm	53
7.5.2	Destination Heading	54
7.5.3	Angle and Direction to Rotate	54
7.5.4	Applying a Proportional Controller	55
7.5.5	Calculating Distance and Checking Arrival	56
7.5.6	Testing the Navigation System	56
8	Limitations and Conclusion	58
A		59
A.1	Survey	60
A.2	Technical Drawings	62
A.3	Waterproofing research	64

1 Introduction

Before the the path of the semester project is determined, we have to pay attention to every single idea, even the one that looks crazy or or irrelevant at first blush.

A brainstorming chart was created to get an insight into each idea suggest for evaluation. In total 15 ideas were listed and described by the following categories : mechanical challenges, electronic challenges, software challenges, components, potential market, cost, machine learning, sponsorship potential, user safety. All the ideas with the description can be seen in Appendix.

Each category consists from 1 to 5 information. It was a qualitative analysis that helped the team to understand application, define potential market and sponsors, narrow the estimations of required time, budget and range of skills. After evaluation of those factors it was decided to choose eight ideas for further quantitative analysis. The choice was based on the relevancy of the idea to the semester project requirements and time required to complete it. not finished or irrelevant projects will significantly decrease score of the project. All five members has to agree on the value, that was assigned to the particular part of the development process. The Justification was to avoid personal preferences and make the process objective.

	weight	Snowcat	Weeder	War	Danfoss	Water	Dog	Labrador	Eva
Originality	20%	5	6	6	1	7	6	8	5
Within budget	5%	1	8	4	8	7	2	8	6
Software	10%	3	2	2	8	7	1	5	1
Electrical	10%	6	5	3	8	5	3	5	4
Mechanical	10%	5	5	5	8	3	2	7	5
Scalability	5%	1	7	1	2	5	2	7	5
Availability	5%	6	5	6	1	7	6	8	5
Demanding	5%	3	3	3	8	3	3	3	1
Relevancy	30%	5	3	4	7	8	5	4	2
Total		66	98.5	72.5	70	118	75.5	111.5	79.5

Table 1: Evaluation of preliminary ideas

The choice was based on the relevancy of the idea to the semester project requirements and time required to complete it. Those two were considered the most important, as both, not finished or irrelevant projects will significantly decrease score of the project. As can be seen in the Table 1.1 idea Water that represents the rescue boat achieved the highest score.

Afterwards the idea was consulted with our supervisor, since the boat was not a standard choice of the students. There were more potential risks compared to land vehicles, since the electronic could easily get in contact with water during testing. After the final decision to build a rescue boat was made an approved by supervisor, a market research had to be done, to get familiar with the solutions already available. In the same time a user research was done to target the potential customers and get their feedback

2 Problem formulation

2.1 Market research

Before the requirements and work-cycle was determined there was an analysis of 4 different companies that are producing unmanned life saving boats or buoys. The analysis is consisting of short description and a table comparing technical specification. All the data will be used during the requirement writing, so that the product is capable of potential competition on the market.

Emily boat It is 127 cm long boat with a buoy covering the top of the boat. It controlled by RC and a rope can be attached to it in order to get it back on shore after being deployed. It has 4 other modifications, police, sonar, man over board and swift water. It is powered by one motor that is also used for steering.

Hover Ark H3 It is Remote controlled Lifesaving buoy controlled by RC with the function of an automatic return in case of lost signal. It has an upgraded version with lights for better visibility. It is capable of transporting the rescued person back to the shore, but he/she has to be able to grab on it.

Orca H9 Live-saving watercraft that is the most powerful from all the products listed. It is a manned vehicle that can carry a single lifeguard and carry or drag the victim back to the shore. It is powered by one water jet engine. Compared to the jet-ski it is more compact and needs less power.

Dolphin 1 The look and usage of this lifesaving buoy is similar to Hover Ark H3. Except that it has an extra camera in the front and is a bit larger.

Water Rescue Stretcher Bed It is the extended and more powerful version of Dolphin 1. Between its two propellers is located the stretcher bed onto which the person can be placed and does not have to hold it during the transportation.

Product/Parameters	Size[mm]	weight[kg]	power[w]	runtime@speed[km/h]	payload[kg]	control
EMILY	1230x355x355	12		13min@37	700	RC+rope
Dolphin 1	1190x850x200	13	1800	30min@12	225	RC
Stretcher bed	1680x730x260	30		30min@15	200	RC
Orca H9	910x53x32	23	4500	80min@16	300	manual
Hover Ark H3	1030x630x20	13.8		45min@18	200	RC+autoreturn

Table 2: Comparison of products

Conclusion After the investigation was concluded that all of the products have to be navigated by person to the victim. Only Hover ark is capable of an automatic return. Therefore there is a space for extended functionality of navigation to the victim or to a lifeguard as an assistance. None of the above mentioned has the ability to connect with the user and track his motion. One of the ideas how to make the autonomous boats interact with people is the possibility to pair it with a smart watch.

2.2 User research

In order to make the boat useful and interesting for the future customers a questionnaire was created. It was sent to lifeguard, whose input provided us with feedback and improvements, that they think, will increase the usefulness of the boat.

The questionnaire addressed particular properties of the boat. Such as the shape, maximum load, maximum speed, operation time, colour and accessories. All the questions are evaluated below. It has a polish and english version and was answered by 5 professional lifeguards. The full version of the survey can be seen in appendix [A.1](#)

2.3 User requirements

Results of the survey were used as a guideline of user requirements. All the questions were made in a way that the requirements can be made directly out of the survey data. Specific time and distance limits, colour, accessories, shape can be easily measured afterwards to check if the requirements were fulfilled. The sum up of the user input gives the feedback of the fact that the autonomous life saver boat can be useful project used in real life.

1. Shape of the boat: the person should be able to grab on it and lay on it, like a stretcher.
2. Accessories of the boat: camera - document the mission, warning lights, first aid kit, warning sound, 5-min oxygen bottle.
3. Help with getting to the drowning person.
4. Holder to the boat for the lifeguard.
5. Colour of the boat: bright, clearly visible colour, regulated by country's regulations.
6. The boat should be able to achieve at least 200 m distance from the shore.
7. The longest time of getting to the drowning person should not go beyond more than 6 min.
8. The longest time the boat should be able to work on the water without recharging should total 30 min.

2.4 Product requirements

After evaluation of user requirements, university requirements, given time and budget, project requirements were created and divided into 2 groups. Need to have requirements are formulated in a way that the final product should be a fully working downscaled prototype. They are giving detailed insight into the technical characteristics and functionality.

Initial version of need to have requirements were adjusted during the development process. For example the obstacle avoidance based on sonar data was moved to nice to have. It was caused by two things. Sonar modules were not within the budget of our semester project. The ultrasonic modules that were provided us by our supervisor, had limited documentation, therefore more time was required. Nice to have requirements are describing the full scale prototype with extensions that fulfill all the user requirements. They are also a guideline for the future development.

2.4.1 Need to have requirements

1. University
 - (a) at least 1 analog sensor
 - (b) motor control driver module
 - (c) designed custom pcb
 - (d) 1000 kr for new component
2. Body and component housing

- (a) Should have 1 handle on the back with a trigger button
Justification: useful when loading person, and detecting if person grabbed it
- (b) Should be painted in a bright visible colour regulated by the law.
Justification: better visibility and law restrictions
- (c) Should contain two waterproof boxes as a housing for components
Justification: damage of the electronics if the water leaks in.
- (d) Should have two motors
Justification: easy way to steer the boat compared to other options

3. Speed and endurance

- (a) Minimum speed to the person should be 1.1 m/s
Justification: maximum distance/3sec
- (b) Should be able to run for 10 minutes without recharging
Justification: The weight and price of the battery
- (c) Should be able to carry 50 kg load on the way back
Justification: the maximum payload 150 kg / 3. Using 1/3 of the motor power

4. Navigation

- (a) Should use a GPS with the precision of 3 m
Justification: price of the models with the highest precision
- (b) Should be able to receive signal in range of 200 m
Justification: largest distance desired by user
- (c) Should have a compass to determine the orientation relative to north
Justification: determine the orientation of the boat

2.4.2 Nice to have requirements

1. Should contain a net stretcher bed
2. Should have one handle on the left and right side
3. Should be able to run for 30 minutes
4. should have inbuilt speaker and warning lights
5. Should be able to house camera, first aid kit and 5-min oxygen bottle
6. Should be able to help the lifeguard get to the drowning person
7. Should be able to detect obstacles based on the sonar data

2.5 Risk assesment

This Risk Assesment is based on Standards applicable for medical devices. Which relates to the project that is report is about. Below mentioned standards are taken into account.

2.5.1 Standards to be satisfied

1. ISO 14971 (ISO 14971:2012)

The development team may self assess and discard any negligible risks for the purpose of the risk assessment.

2. Directive 93/42/EEC

This standard is above ISO 14971 in order of application.

All risks must be minimized, regardless of size, and balance against benefit of the device. Only nonacceptable risks need to be accounted in ISO 14971.

3. Risk Reduction

(a) I14 - risks need to be reduced "As Low as Reasonably Practicable", with economic considerations permitted (ALARP).

(b) D93 - risks reduced "As Far as Possible", and no economic considerations permitted.

(c) I14-6.5 - If residual risks are not acceptable and further risk control not practicable, assess: Does benefit outweigh risks.

2.5.2 Risk Management

1. For the purpose of this project, the only applicable part of the life cycle where any risks are great enough to warrant the application of this plan will be during the "intended use time". This time is defined as from the beginning to the end of the demonstration of the project, whether digitally recorded or live.

The process: the device is meant to be a life saving boat that helps the lifeguard to take the drowning/injured person in the water back to the shore. The boat is called by a lifeguard's bracelet, comes to the lifeguard position and collects them and the injured back to the shore.

2. Responsibilities and authorities of the project. All group members are meant to work on each part of the project at least in very low level. Each person is responsible for explaining their done tasks to other group members. Each part of the project has a supervisor, that is responsible for giving tasks and checking:

(a) Management: Vojtech Ilcik, Zuzanna Parnicka

(b) Electronics: Al Muthanna Almoslem

(c) Programming: Hritik Roy Chowdhury

(d) Mechanical: Yieng Liu

3. Reviewing tasks

Each risk is assessed on rolling basis as new and possible hazardous situations are identified throughout the project design and testing phases. The risk's acceptability (including risks with unpredictable probability of occurrence and those of which harm cannot be estimated) will be assessed using the criteria matrix from Appendix (Figure X).

(a) Waterproofing failure.

Primary compartment water leakage can cause boat drowning, loss of integrity and secondary compartment water leakage which would cause electronic components damage. The primary compartment water leakage is not necessary to cause harm to the user. However, the secondary compartment water leakage would be a harm to the user, and could result in temporary impairment requiring professional medical intervention, due to electronics having contact with water.

Solution to prevent water leakage:

- i. Use of silicon on all the possible water leakage corners, holes.

- ii. Testing the primary and secondary compartment in different conditions and times to see if/how the water leakage is changing or is it even appearing.
- iii. Choosing as many waterproof components as possible.

Risk Assessment: Occasional - Serious.

(b) Navigation system failure.

Navigation system consists of: GPS module, magnetometer and accelerometer. The failure could be caused by interfering magnetic fields, which could cause the magnetometer to not calibrate properly.

Solution to prevent the navigation system failure:

- i. Putting navigation components as far as possible from devices creating magnetic fields.
- ii. Findign a solution for distance beetwen servo and magnetometer.
- iii. Testing in environments with different magnetic fields and noticing impact on the system.

Risk Assessment: Occasional - Negligible.

(c) Communication failure.

RF module in the project is meant to send and receive coordinates of the bracelet. The problem could be the distance. The module specifications need to fulfill the product requirements. If there was a problem with distance it might be that the boat does not receive location it should go to. Another problem could be that the RF signal is interrupt by environment.

Solution to prevent communication failure:

- i. Choosing RF module in respect to user requirements.
- ii. Testing RF in different circumstances.

Risk Assessment: Occasional - Negligible.

(d) Motor failure.

If the motor fails, the boat is not able to drive. Therefore, the goal of the project is not satisfied. The boat cannot get to the bracelet location or help to get back to the shore. The motor failure can cause damage to limbs, which is harmful to the user. Motor failure can be caused by overheat, kelp/rocks, seaweed getting stuck in the propellers.

Solution to motors failure:

- i. One of the best solutions is choosing motors that will be able to satisfy product requirements and at the same time, choosing the motors that are less possible to fail.

Risk Assessment: Remote - Minor.

(e) Battery overheating or explosion.

Battery overheating or explosion can be caused by overcharging, damage, exposure to high temperature. Battery overheating or explosion could be a harm to the operator. It could cause serious injury or less serious harm. However, it is hazardous to life of the product or the user.

Solution to battery overheating or explosion:

- i. The battery will be cooled by being in the water.
- ii. Overcharging the battery needs to be taken into account, while preparing and testing the boat.
- iii. Overall damage can be prevented by being carefull with all the electronics components.

Risk Assessment: Remote - Serious/Critical.

(f) Unexpected electronic components failure.

Unexpected electronics component failure can be anything. E.g. overheating, damaging the component by accident. It is not possible to find solutions for this problem. In the testing phase, components can be replaced or repaired. However, when something happens while the working process, only the emergency stop can help. However, this issue is possible to prevent/make it less possible to appear thanks to many tests. Risk Assessment: Remote/Occasional - Negligible.

(g) Failure of safety features.

Safety features for the project are: emergency stop, handle for the user. It might be that the emergency stop does not work for unknown reason during the work cycle, even though it worked while testing. The same can happen with the handle. The handle is made to let the program know, if the user is holding the boat, if the handle is not held the boat stops moving and waits for the signal. If this feature stops working, the boat could drive without the user and leave them somewhere on the water, or harm the user. Solution to failure of safety features:

- i. Having more than one safety feature.
- ii. Testing safety features in different environments.
- iii. Waterproofing safety features.

Risk Assessment: Remote/Occasional - Minor/Serious.

(h) Management problems.

Management problems can appear as not sufficient time management, uneven tasks distribution, miscommunication in the group, budget overrun, need to wait for some tasks to be done to be able to start other tasks.

Solution to management problems:

- i. Choosing group leader.
- ii. Choosing leader of each project part (electrical, mechanical, programming).
- iii. Making a timeplan in the beginning and adjusting it during the project work.
- iv. Making a budget plan.
- v. Using recycled components from university, home, companies.

Risk Assessment: Probable - Negligible.

4. Verification activities

Verification of the above data is done via data from manufacturer, data from typical operations of similar purpose and calibre, as well as data generated while testing the device itself. Figure X in Appendix.

5. Review of production and postproduction information. Because the product has a very short life cycle, this section is not applicable. However, data retained from any testing during the design and prototyping phase will be reviewed and used for risks.

3 Theoretical analysis

3.1 Component selection

3.1.1 What to consider before choosing GPS module

List of needed requirements	
Type of requirement	specific of the requirement for the project
size wristband	max 30 mm x 30 mm
size boat	max 60mm X 60mm
update rate	1-10 Hz
power requirements	3.3/5V - we make a voltage regulator
number of channels	?
time to first start	?
antenna	best if included
accuracy	the more precise the better, budget limitation , at least 3 m horizontal accuracy
microcontroller compatible with	Arduino

Table 3: GPS modules requirements

3.1.2 Considered options and their specifications

1. Spark Fun RTK-SMA

The SparkFun RTK-SMA GPS module is very precise, up to 0.01m in horizontal accuracy. However, the price is 250 \$ which is out of the semester project budget. The module is advanced and can do RTK. However, it is not needed for the prototype of the project. [\[1\]](#)

2. NEO - 6

Due to its low price relative to the functionalities it offers and compatibility with Arduino, this module is indisputable choice for those who want to learn how GPS works. This module is based on NEO-6M chip from U-blox. It has a Power Save Mode that makes it suitable for a wristband locator. It is also the smallest chip among the others listed in a [Table 4](#) It includes antenna with sensitivity patch of 161dBm.

3. BN-220

The needed requirements are fulfilled despite the price. Which for 1000 kroner budget is too high if the shipping price is added. [\[2\]](#) [1](#)

requirement	Spark Fun RTK-SMA	NEO - 6	BN-220
size	X	X	X
update rate	X		X
power	X		X
desired number of channels			72
time to first start (cold/warm)	-148dBm/-157dBm		26s/25s
antenna			
accuracy of min 3 m	X	X	X
compatible with Arduino	X	X	X
budget	not in the budget		about 150 kroner but expensive shipping

Table 4: GPS modules comparison

What waterproof options are possible for the project 3 It was decided to use silicon. It turned out to be the most suitable option for our purpose. The whole research about waterproofing options is in Appendix.

4 Project Management

4.1 Stage gate model implementation

This management model was chosen due to its clear and chronological distribution of the project work along the whole semester project period. It acted also as a guideline for report writing, when each gate symbolizes one chapter in the report. Time plan was as well developed with the help of this model, when a deadline was assigned for each gate and time period for each stage. The stage was further divided into days and their tasks.

Each gate was consisting of requirements. If they were not met, project could not proceed. If the product could not fulfill the requirements both stage processes and gate requirements were evaluated. For example the requirement in a gate was to avoid obstacles based on the data from the ultrasonic sensor. Since we initially overestimated the task due to a limited documentation, the requirement had to be changed. We were only capable of building electrical circuits for them. Below is the sstr

4.2 Task management

For the purpose of this project two communication work division and progress tracking a planning platforms were chosen.

Notion was used in the during the problem formulation. It has the functionality of creating databases and progress tracking. Preliminary ideas were listed here and evaluated. It was keeping track of our learning process in LaTeX, Notion and other software needed for the semester project.

Microsoft teams was used from the designing phase until the end of the project period. Compared to notion it was capable of file storage. I was usead as a platform for video-calls. Its planning extension was used to sort all the task according to its respective area and assign people who were responsible for completing

To gave each member a better overview of the whole upcoming task and deadlines a timeline was created on the blackboard in our project room. We have divided the whole time period into weeks. For each week a set of task was listed and approved by all the members than further distributed into Teams.

During the first month of the semester project we were quite bad at at tenting team meetings on time. We got an inspiration from other group and sign a cake agreement. Everyone that was late to the meeting have to bring a cake.

5 Mechanical design

5.1 Boat's components housing

The boat should be waterproof. However, there should be a second waterproofing layer. Therefore, two boxes were made for the insides of the boat. They were laser cut from acrylic 3 mm, connected with 3D printed with PLA brackets figure [1](#) and then the connection between the parts was filled with silicon figure [3](#) and [4](#).

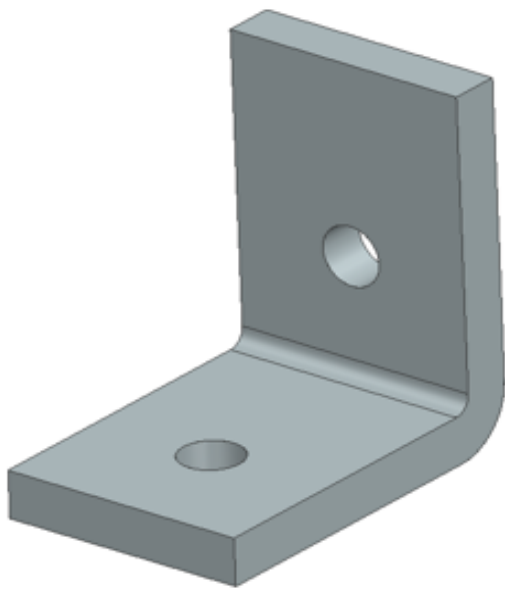


Figure 1: Bracket 15 × 15 CAD Design

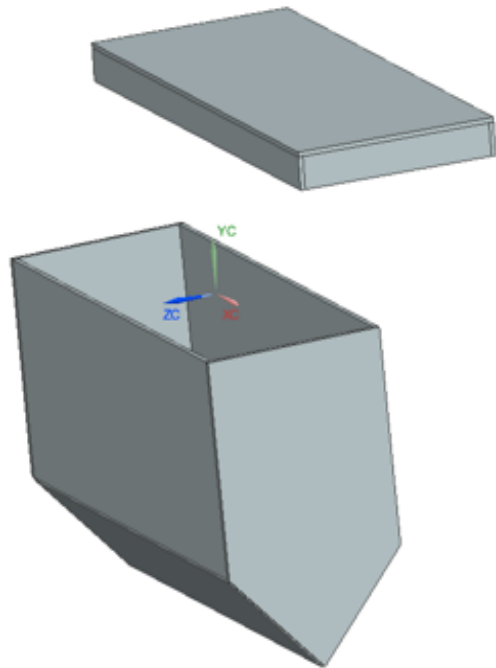


Figure 2: Version 1 of the box



Figure 3: Silicon sealed box - Side view



Figure 4: Silicon sealed box - Side view

The top of the boxes had two versions. The one in figure 2 was supposed be stuck on the bottom part. However, after testing it turned out it would not secure the components from the water when it was flipped.

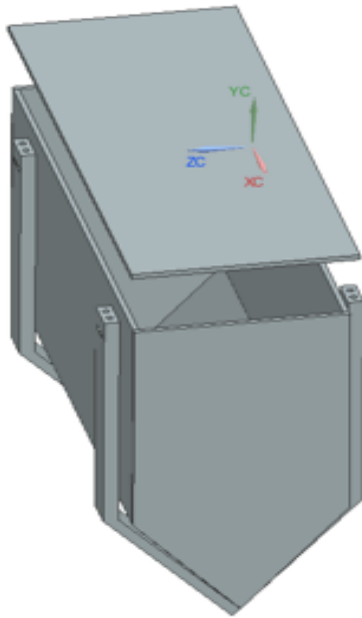


Figure 5: Version 2 of the box

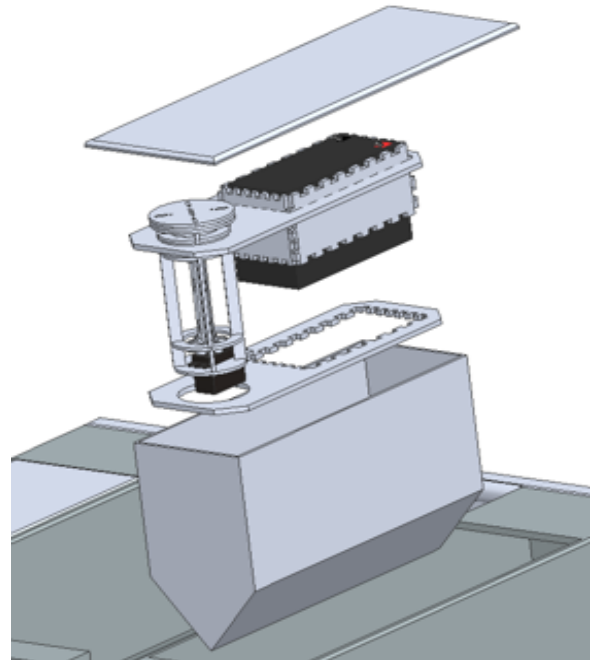


Figure 6: Assembly of the tower housing block

Therefore, the version number two depicted in figure 5 was designed. It was designed to be a plain rectangle screwed to the 3D printed frames. The frame is a tight fit to the box, and the top plate is screwed into the 3D printed from PLA frame.

Inside each box there has to be a battery. In one of the boxes there have to be rest of the electrical components. To fit the battery and tower inside the boat, two plates were laser cut from 5 mm acrylic . They will be put in both boxes and prevent components from moving. One of the boxes, while testing the waterproofing, started leaking. The second one did not, even though both of them were done the same. Therefore, one of the boxes had to be done from the beginning. Afterwards, it was tested again and the test was successful.

5.2 Calibration Tower

The calibration tower is a mechanical structure built for boat's instruments' concentric automatic calibration.

5.2.1 Calibration needs

The need of a calibration tower originally arises from issues encountered in the magnetometer's software. The magnetometer's output vectors form a circle centered at random positions within a cartesian coordinate plane. To determine the center of the vector circle at startup, the magnetometer must be moved with at least two degrees of freedom before starting motor functions. This is so that the subsequent positions of the directional vectors could be found relative to the center of the coordinate system. The controlled movement of the magnetometer requires a mechanical holder that allows sensor to rotate precisely in a circle. Due to possible additional sensors from later projects, the holder needs to be able to scale to accommodate more electronic instruments.

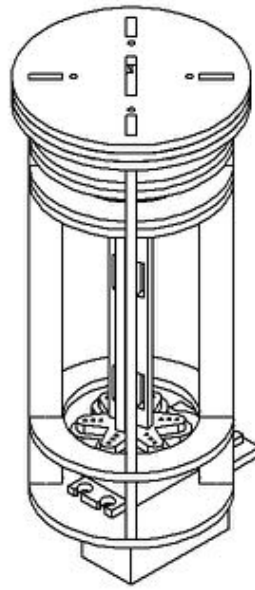


Figure 7: Calibration Tower Assembly

During preliminary testing of prototypes, the magnetometer's readings are uninterpretable when taken close to any reasonably strong electromagnetic fields, such as the servo motor used for the calibration itself. It is therefore determined that the sensor needs to be 20cm away from the servo motor to avoid interference. This number is later changed to 12cm resulting from more precise testing done because of the waterproofing compartment's dimensional limits.

5.2.2 Instruments installation and accessibility

Three of the main risk factors considered for the tower's operations are structural stability, repair & accessibility, rust resistance. The risks are balanced according to the risk assessment criteria

Table 5: initial risks breakdown

Probability	Severity				
	Negligible	Minor	Serious	Critical	Catastrophic
Frequent					
Probable				Rust	
Occasional					
Remote				Structural breakdown	
Improbable	Repair & Accessibility				

Structural stability risks are minimized by creating an integrated tower that contacts the main support structure in multiple, redundant positions, allowing for only one degree of freedom of least possible movements, reserved only for extraction. The moving members of the tower are constrained using concentric circular slots to ensure maximum contact between the spinning disks and the fixed boat structures to only allow for rotational movements. To avoid

tangling, moving wires are threaded through the rotating shaft to the base of the tower and given just enough slack to allow for 180 degrees rotation.

Repair & accessibility The structure is designed for easy dismemberment and component replacement to two different extents. Top level disassembly of the tower requires no special tools or complicated maneuvering of mechanical components. The user simply takes any piece of flat material of reasonable rigidity such as a home key, insert the flat edge into the top and slightly twist the lock to unlock the tower top. From there, the user can simply unplug the magnetometer from wires that have been fixed in the mechanical socket slot. If required, simply lift the tower by its locking lid, and the entire structure will slide out of the tower slot, and upon the same unlock motion, the tower can be entirely dismembered into its constituent parts. Designs for repair & accessibility are made to accommodate scaling of number and sizes of components. The tower dimensions are easily modifiable, and more slots can be added for any components of similar footprints and connection requirements.

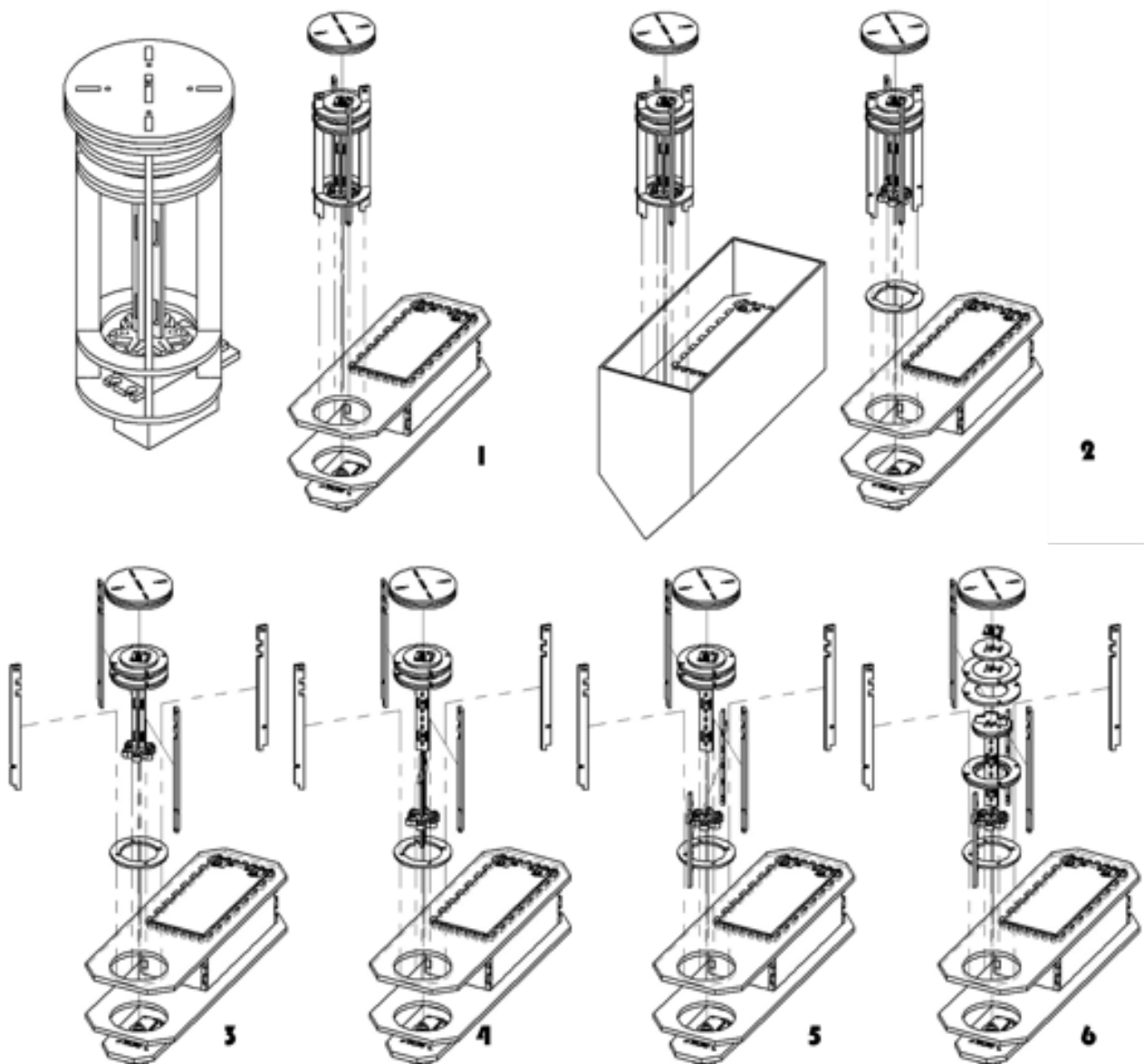


Figure 8: Compass Tower Explode Stepped

Salt-water resistance All components of the tower are designed to be manufactured with acrylic and not materials that may expand with salt water, such as metals and wood (MDF). This includes the component thickness as well as their specific ability to flex. This is especially reflected in the design of the locking mechanism and the interlocking rotating shaft, where MDF will fail due to mechanical stress if cut to the same laser-cutting pattern as the acrylic design. This is determined experimentally.

5.2.3 Design process

Overall Design The design process uses an inductive approach to minimize the waste of space in the waterproof compartment. Using the least possible distance and the width of the compartment, the maximum size of the tower is deduced. With the principle of not compromising the distance between the sensor and the motor, the components are spaced as close as is deemed safe by testing.

Lock mechanism The lock mechanism requires spring tension to minimize the possibility of a slippage, which will likely result in the tower's malfunction. To circumvent the use of small metal springs that maybe corroded on long term standby, the lock is designed with a compliant mechanism sandwiched between two supporting pieces so that the lock itself does not sustain any lateral or rotational forces but allow them to be distributed onto the slots of the sandwiching pieces, ensuring minimal load onto the compliant springs. The main difficulties in the design arise from the limited dimensions and flexibility of acrylic as a material. The working version of the lock mechanism ensures the flexed bends are of even thickness throughout the spring's body as well as while it is in motion. Each bend is spaced using circle packing to evenly distribute spaces between the uneven spaces due to the lock circle's growing circumference. A circular relief is then cut into the joints to distribute the tension evenly around the bend arc. Without taking these measures, the bends are extremely prone to cracking because of uneven material stiffness and the resulting uneven bending and force distribution. For using laser cutting, the drawings accommodate for the width of the laser and uses it to provide spacing between the spring and its guiding curves. A tight fit is likely to result in the spring sticking to the sides. The lock springs are designed through five iterations, resulting in one with the desired proportions.

5.3 LED lamp

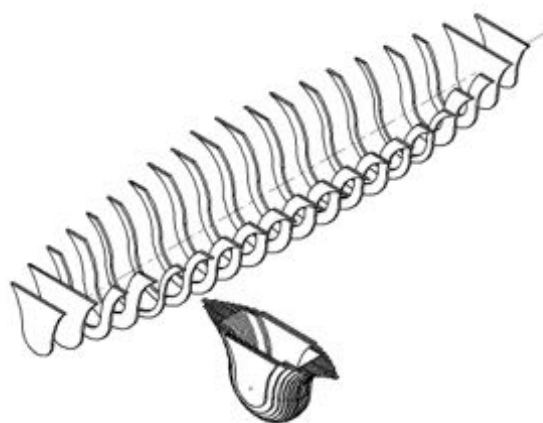


Figure 9: lamp

5.3.1 Illumination needs

The LED lamp is created to illuminate the boat's relative position to the drowning party to facilitate rescue. Due to the possibility of the rescue occurring at night, there is a need for the drowning party to easily locate LiSa. The

boat may operate under such risk for a considerable amount of time during its lifetime, which increases the risk of the drowning party not able to identify the boat's location. The boat's GPS is only accurate up to a 5 meters range, so it is necessary for the boat to be highly visible, in case of possible impaired vision, reduced visibility due to fog or rain, or the drowning party being partially under water at the time of the boat's arrival.

5.3.2 LED design

The LED selected comes from an 8000-lumen car light LED with active air cooling. After testing, it is determined that the light is also suitable for underwater operation at least over 2 minutes with active cooling system on. This is tested in fresh water, so the LED's ability to withstand salt water is unknown. Due to visible exposed circuits and metal, the design opts for an oil immersion design for a balance between waterproofing and heat dissipation. The transparent container for the LED protects the bulb from salt water while allowing the active air-cooling system to run in the oil, dissipating heat without contact with the outside environment. Two of the main risks that pertain to the presence of an oil immersed LED are oil leakage, visibility, and the potential for objects or people to accidentally contact the lamp. To minimize this risk, the LED container is completely sealed up to the wire leading to the bulb. The container is also placed up-side-down, with its oil containing vessel facing up to physically limit the possibility of oil spillage. The vessel is tightly fastened onto the boat, and shaped organically to minimize damage both to the boat and to any object it encounters, most notably the possible drowning party. The position of the lamp is also facing the surface of the ocean towards the drowning party to increase visibility. The laminated acrylic disperses and refracts the LED's light so that only temporary discomfort may result from looking directly at it for the duration of the rescue.

5.4 Handlebar

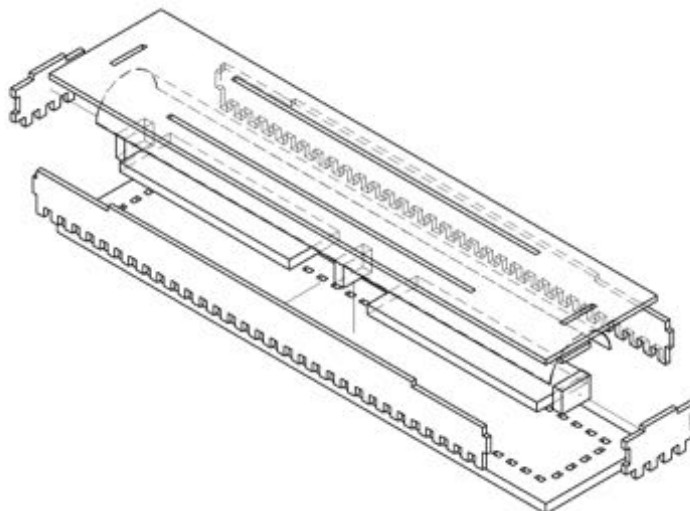


Figure 10: handlebar

5.4.1 Handlebar requirements

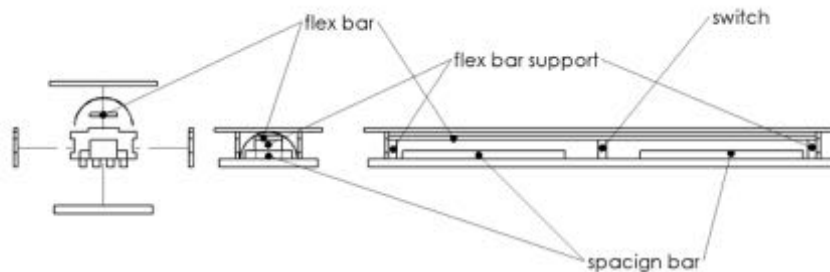


Figure 11: button bar mold internal components

To detect and determine when the party being rescued has latched onto the boat, a safe, intuitive, reliable approach is needed. These requirements resulted in a soft, silicone handlebar with an embedded end stop switch. The handlebar is placed within the safety of the semi-enclosed center space of the boat, with silicone that resist slight touches to the button but is extremely sensitive to being grasped. To ensure the grasping force can trigger a button press at any point on the handlebar, an acrylic strip is embedded between the front silicone casing and the button, with its two ends fixed on two embedded supports. When grasped at any point, the acrylic bar deforms to nearly the same spot in the center. Till today, this always triggers a break in the circuit, indicating a button press. To ensure the rescued party is brought to safety, the boat is programmed to only run its motors only when the bar is depressed on its returning journey.

5.4.2 Handlebar manufacturing

The handlebar is formed by layered silicone casting. First, the front layer of the silicone is filled and partly cured to ensure stability and adhesion. The acrylic bar is placed on top of this layered, then covered with acrylic. Then, two spacing bars, two support bars and the space bar for the switch is placed on top. The spacing bars are there to reserve space for future construction. The future placement of a switch requires a spot to be reserved so that the switch can be carefully placed in without being depressed permanently by the curing silicone. The other two spacing bars reserve two air pockets that ensure the bar can be easily depressed without the use of too much force. The spacing bars are taken out and their respective air pockets filled or covered as needed. The bar is then taken out of the mold and fastened onto the boat handle with two bolts. A piece of MDF is also embedded into the handlebar with two holes reserved for the bolts to ensure even force distribution around the area being penetrated and fastened by the bolts.

5.5 Bracelet

The bracelet is the part of project that is a cover meant to secure the PCB and be worn by a lifeguard.

Short description about the idea behind (to learn more read about the PCB design for the bracelet in electronics chapter):

Bracelet is worn by the lifeguard while swimming to the drowning person and on their way back to the shore. The button in the bracelet is pressed by the lifeguard, then the coordinates of the lifeguard are sent to the boat.

The bracelet should be as small as possible. It cannot handicap the lifeguard during swimming as well as during

saving the drowning person. The requirements for the project were to design a PCB. The bracelet described in this report is a prototype.

The prototype is too big to be used normally. However, it was designed to be as small as possible in respect to the PCB and battery size. The bracelet is not main part of the project and is the "nice to have" part of it. If the project would be developed, the PCB would be created from smaller components, that are usually more expensive and had to be ordered. Like small SMD components, only a chip of a microcontroller, e.g. not the whole Arduino Nano, but only ATmega328p.

The design consists of 4 main components:

- Top cover.

This is the part printed from TPU. It makes pressing the button from the PCB possible. Also if anyone hits the top of the bracelet it makes less harm, than PLA would. The 3D printing is not 100% waterproof, so the idea would be to cover it by silicon, resin or another waterproofing paint etc.

- Bottom cover.

This part had to be done from harder than TPU material, because it has to have the watch bracelet attached to it. It makes it not as safe as needed, but because it is a prototype it is acceptable. The 3D printing is not 100% waterproof, so the idea would be to cover it by silicon, resin or another waterproofing paint etc.

- Acrylic rectangle.

This part of the bracelet is supposed to pressure the TPU to the PLA. Since the TPU top cover is not strong/hard enough to be able to make the connection of both top and bottom covers waterproof.

- Bracelet.

The bracelet for the hand is taken from a watch found in one of the group members house. The CAD design is adjust in respect to its size.

In the first design in figure 12 the Acrylic rectangle was 3D printed from PLA, but because of the size of it, it was breaking and was not hard enough to press the TPU onto the PLA bottom part. Another problem with the first design was that the rectangle was too small, therefore it was decided to make the bottom cover wider as well as the rectangle. And that the rectangle should be done from acrylic.



Figure 12: Bracelet first version

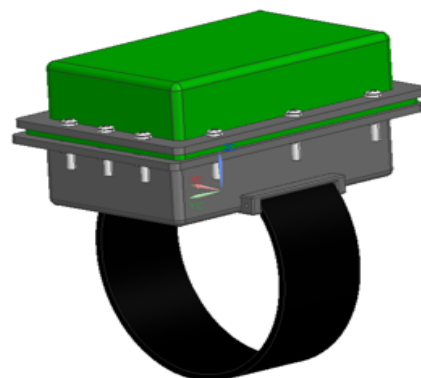


Figure 13: Bracelet CAD design

All in all, the bracelet was not finished due to lack of time. The PCB was done successfully, and the bracelet idea is working as meant to be. The design in CAD, figure 13 was done and theoretically would be working. Therefore, it can be said that the bracelet was halfy done.

6 Electrical design

The Project "Life Saver Boat" comprises two main PCB's that communicate with each other. Each PCB includes various circuits.

6.1 Bracelet PCB

Located on the user's wrist, it includes:

- Arduino Nano
- HC-12 SI4463 Transceiver
- NEO-6/7M GPS Module
- Voltage regulator circuit
- 7.7V Battery
- User button

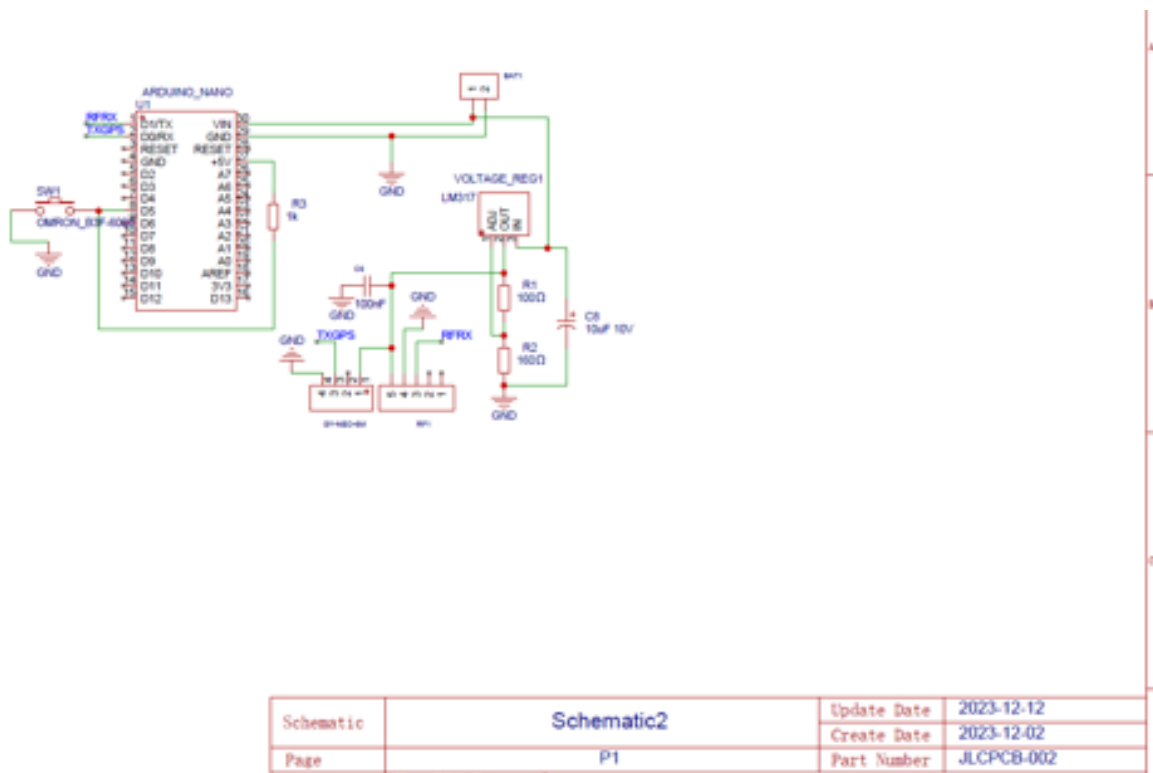


Figure 14: Bracelet Schematic

6.2 Power Supply

6.2.1 Power supply design

The circuit is powered by a 7.7V LiPo battery, which directly powers the Arduino and, through an LM317 voltage regulator, steps down to 3.3V to power up the HC-12 and NEO-6. This voltage regulator was chosen due to its adjust-ability of V_{out} . Voltage output is determined by the resistors $R1$ and $R2$ as per the formula:

$$V_{out} = 1.25V \times \left(1 + \frac{R2}{R1}\right) \quad (1)$$

$$3.3V = 1.25V \times \left(1 + \frac{160\Omega}{R1}\right)$$

Selecting $R2 = 160\Omega$, for a 3.3V output, solving yields $R1 = 97.6\Omega$. As 5% tolerance is acceptable in this case, $R1$ is chosen to be 100Ω with a 5% tolerance.

The power dissipated by $R1$ and $R2$ is calculated as follows:

$$P(R1) = \frac{1.25^2}{100\Omega} = 15.6mW$$

$$P(R2) = \frac{(3.3 - 1.25)^2}{160\Omega} = 26.5mW$$

$R3$ is a pull-up resistor, ensuring that D5 is set to a low digital level when SW1 is pressed. Upon activation, the coordinates of the bracelet will be transmitted to the boat via the HC-12, initiating the process (further details in the Software section).

6.2.2 Power supply test

The test was done using the values during designing process using equation [1](#):

$R1$ - set resistor, was chosen to be 100Ω

$R2$ - adjustable resistor

$V_{out} = 3.3V$, later measured to be $3.27V$

$V_{battery} = 7.51V$ (measured)

$R2 = 160\Omega$

To check if the circuit works as needed, load resistor is added. The load resistor is expected to act as two modules that would be attached to voltage regulator output.

To find the R_{load} , maximum currents needed by modules are found in their data-sheets and then add 20% for assuring.

$$\text{GPS: } I_{\max} = 100mA + 20\% = 120mA$$

$$\text{RF: } I_{\max} = 13.5mA + 20\% = 16.2mA$$

Adding them:

$$120mA + 16.2mA = 136mA$$

$$R_L = \frac{V}{I_{\max}} = \frac{3.27V}{136mA} = 24.0444\Omega \quad (2)$$

The resistor available in the lab was 24Ω and measured to be 24.2Ω , which is within the 5% tolerance.

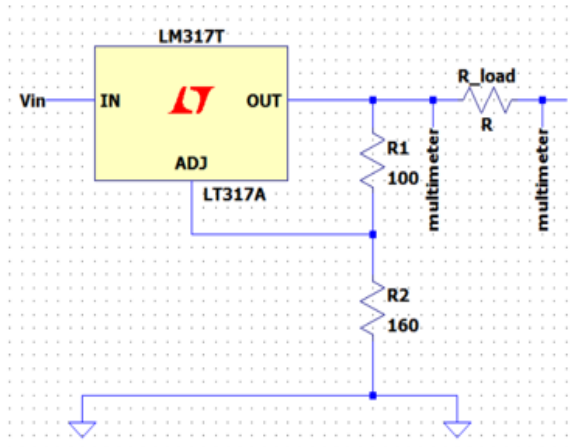


Figure 15: Schematics of voltage regulator test

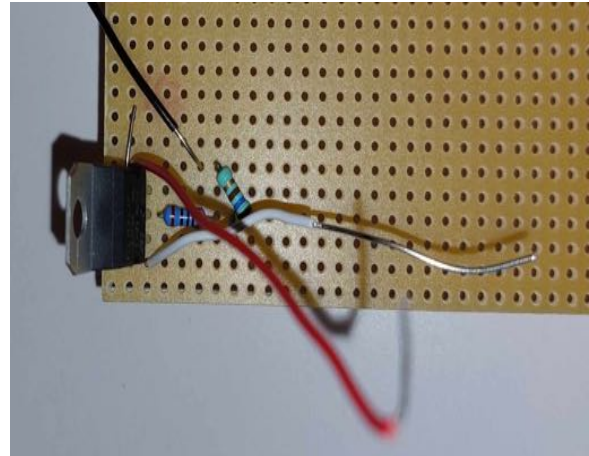


Figure 16: Voltage regulator prototype

After soldering the components, there was a short circuit. However, with use of multi-meter it was easily found and repaired. The voltage output was as required (3.27V) by the modules, therefore it was implemented into the PCB design

6.3 PCB Design

The PCB's size is minimized for wrist wearability, with future iterations aiming for even more compact designs using an AT-mega328 directly on the PCB and SMD components. The PCB has been designed to meet the specifications provided by our team and takes into consideration the capabilities of the SDU CNC machine we're using. We've ensured that there are no vias in the design, and the minimum track thickness of 2.5mm is accounted for, in line with the machine's requirements.

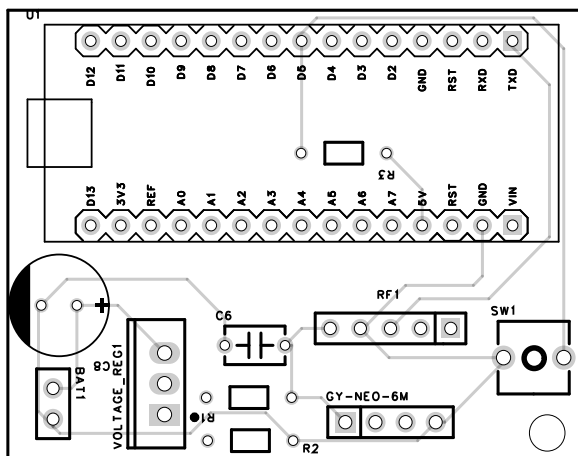


Figure 17: Bracelet first version

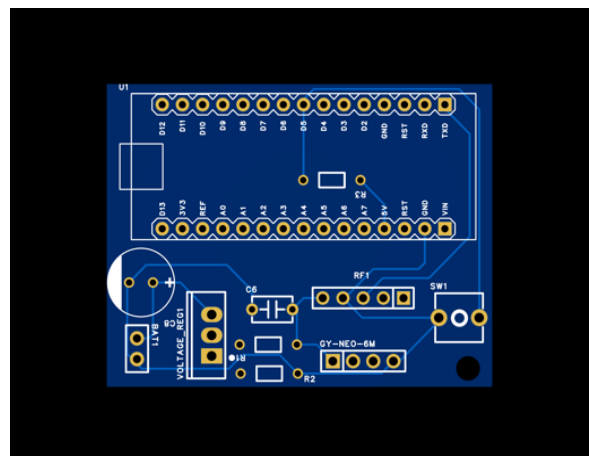


Figure 18: Bracelet CAD design

6.4 Boat PCB

Located in the boat, it consists of 3 main circuits:

- Transducer receive circuit
- Transducer transmitter circuit
- Power circuits

Hardware:

- Arduino Mega
- HC-12 SI4463 Transceiver
- NEO-6/7M GPS Module
- Servo motor
- Compass
- Gyroscope

6.5 Transducer RX Circuit

The Rx circuit consists of an MFB Band-Pass-filter, Amplifier, and Peak Detector.

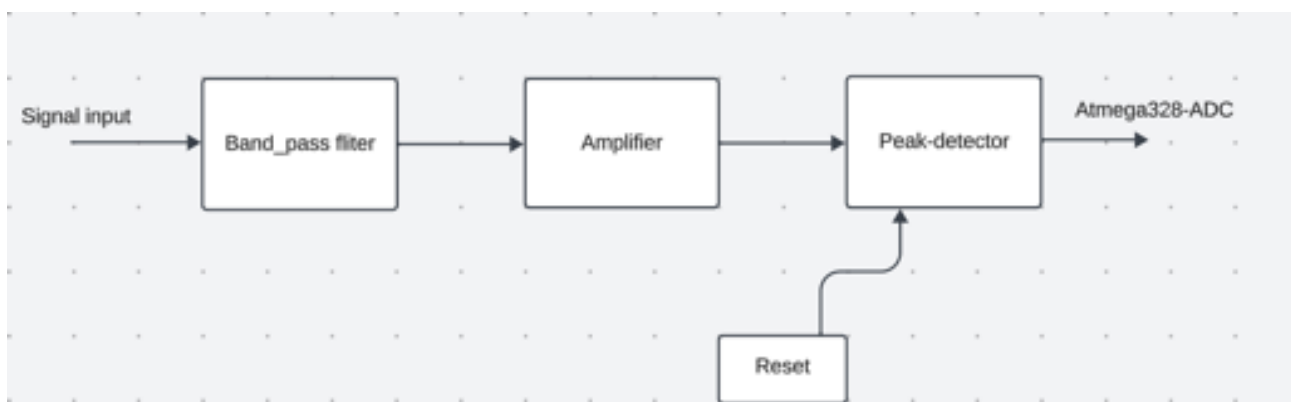


Figure 19: Block Diagram of the transducer receiver circuit

When choosing an operational amplifier (op-amp) for signal processing tasks, understanding its specifications, such as slew rate and bandwidth, is crucial. For a signal with a peak voltage of 2.5p V at a frequency of 43 kHz, the minimum slew rate is a key factor in preventing signal distortion.

The formula for the minimum slew rate required is:

$$\text{Slew Rate (SR)} = 2\pi fV \quad (3)$$

By substituting the frequency $f = 43\text{kHz}$ and the peak voltage $V = 2.5\text{V}$, the minimum slew rate necessary for our application is approximately $0.675\text{V}/\mu\text{s}$. This indicates how fast the op-amp's output needs to change to accurately follow the input signal's fastest transitions.

The OPA4134 is a good choice for our application because of its high slew rate of $20\text{V}/\mu\text{s}$, and its bandwidth, typically around 20 MHz, which ensures it can handle signals well beyond the 43 kHz of our application. It is a solid choice for dealing with our 43 kHz, 2.5 V peak signal.

A practical approach to determining the minimum slew rate required involves moving one frequency decade when calculating the slew rate; in our case, starting from 430 kHz. This method suggests that the minimum slew rate needed is approximately $6.75\text{V}/\mu\text{s}$, which falls within the operational range of the OPA4134.

6.5.1 MFB Band Pass Filter:

Purpose of Design The primary goal of this MFB Band-Pass filter design is to selectively filter out noise and allow the transducer signal at 43kHz to pass through clearly. This is crucial in applications where signal clarity and noise rejection are paramount, such as in sonar systems or precision sensing environments. With a bandwidth (BW) of 6kHz, the filter is finely tuned to isolate the signal within a narrow frequency range around the center frequency (F_c).

Key Requirements

1. Center Frequency (F_c): 43kHz – This is the target frequency of the transducer signal that must be passed through with minimal attenuation.
2. Bandwidth (BW): 6kHz – Defines the range of frequencies around F_c that will be allowed, ensuring that the filter is neither too broad to let in noise nor too narrow to exclude the signal's harmonics or variations. 6kHz is also the BW of the transducer motioned in its datasheet.

Design Calculations A quality factor (Q) is critical to determine the selectivity of the filter. For our application, the Q is calculated as follows:

$$Q = \frac{F_c}{BW} = \frac{43kHz}{6kHz} \approx 7.167 \quad (4)$$

With this Q factor, the filter will have a sharp response, allowing for effective noise rejection while maintaining the integrity of the 43kHz signal.

It's an engineering choice to select capacitor sizes (C_1 and C_2 are chosen to be 10nF), the resistor values are calculated to fine-tune the filter:

1. **Resistance R1:** This resistor impacts the gain and quality factor of the filter. The calculation is as follows:

$$R1 = \frac{Q}{C \cdot \omega_0 \cdot K} \quad (5)$$

Substituting the values, we get:

$$R1 = \frac{7.167}{10nF \cdot 2\pi \cdot 43kHz} \approx 1.326k\Omega$$

Note: The output impedance of the Transducer_B (R_x) is determined to be 840Ω. It's important to account for this resistance as it can impact the filter's gain. Therefore, when calculating the value of $R1$, one must consider the total impedance, which includes both $R1$ and the output impedance of the R_x . Specifically, if $R1$ is designed to be 1.3kΩ, then the actual resistance should be the sum of $R1$ and the transmitter impedance ($Tr_impedance$), equating to 1.3kΩ. Given that the $Tr_impedance$ is 840Ω, $R1$ should be chosen to be 460Ω to maintain the total resistance at the intended 1.3kΩ.

2. **Resistance R2:** This resistor primarily affects the high cutoff frequency (F_h). It is calculated using the formula:

$$R2 = \frac{Q}{\pi \cdot F_c \cdot C} \quad (6)$$

By substituting the known values, we get:

$$R2 = \frac{7.167}{\pi \cdot 43kHz \cdot 10nF} \approx 5.305k\Omega$$

3. **Resistance R3:** This resistor influences the low cutoff frequency (F_l) and the overall Q factor. The calculation is given by:

$$R3 = \frac{Q}{\omega_0 \cdot C \cdot (2Q^2 - K)} \quad (7)$$

With our values, we find:

$$R3 = \frac{7.167}{2\pi \cdot 43kHz \cdot 10nF \cdot (2 \cdot 7.167^2 - K)} \approx 26.33\Omega$$

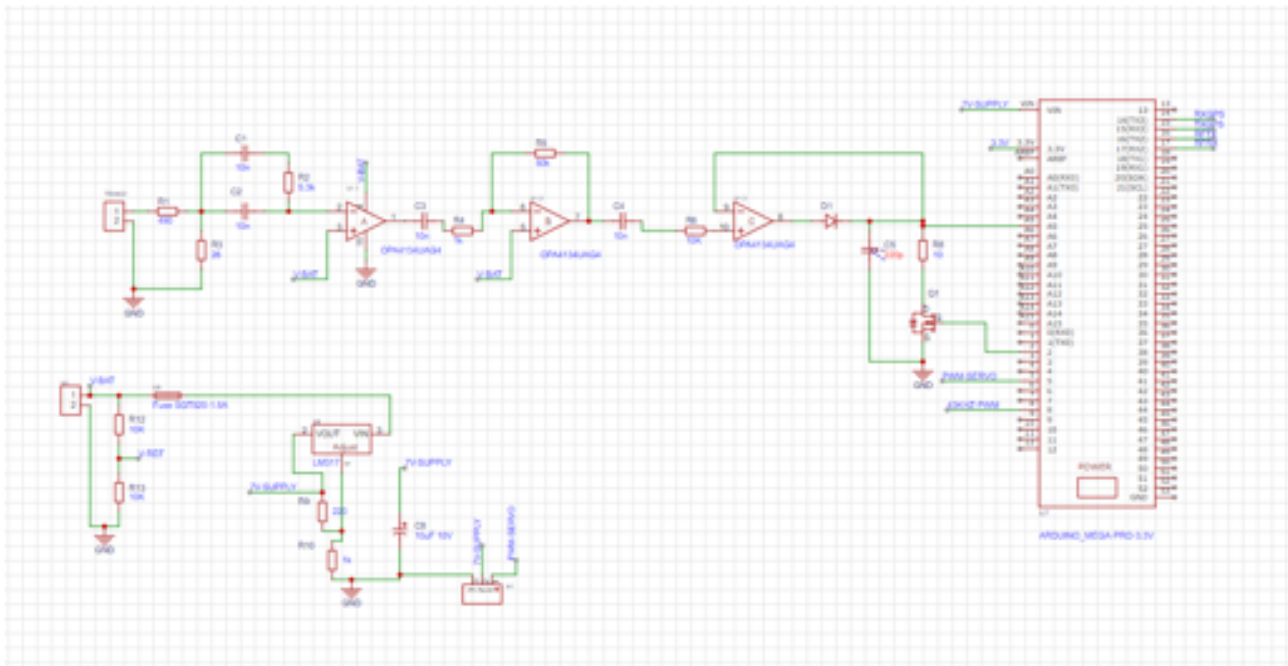


Figure 20: Rx schematic

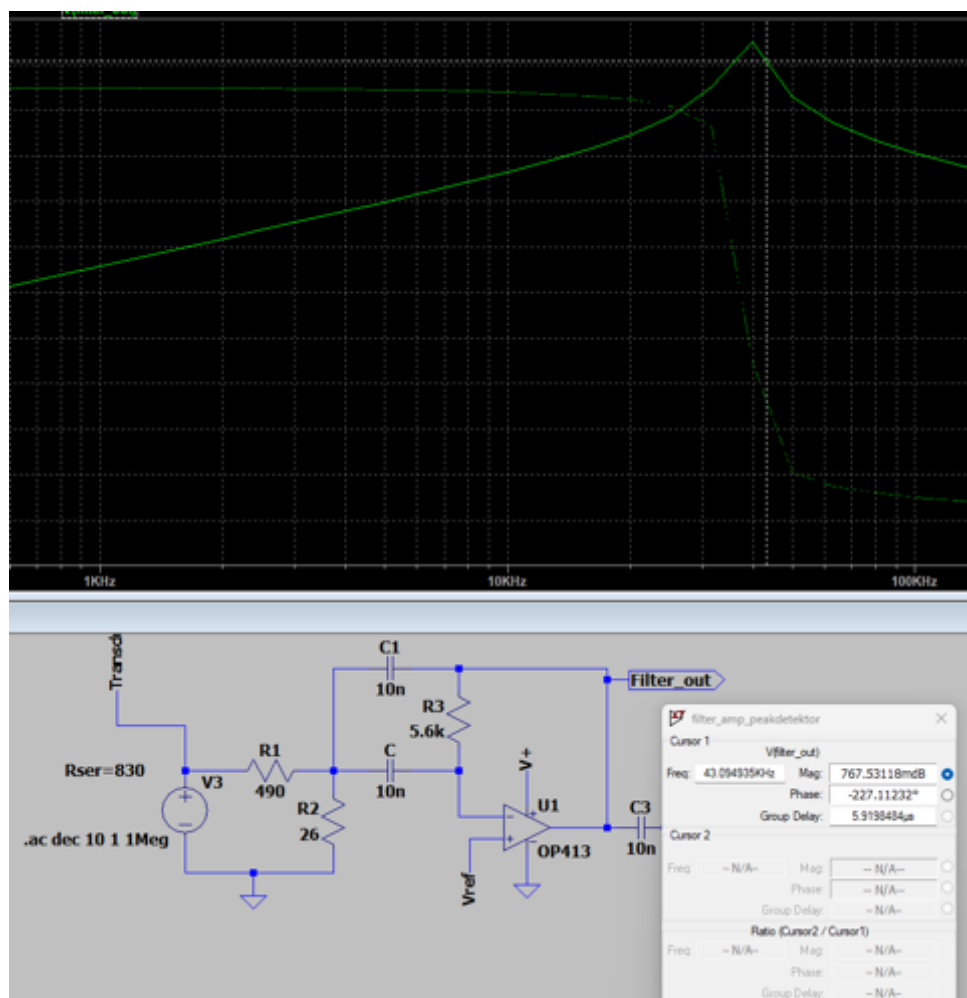


Figure 21: 43KHz location

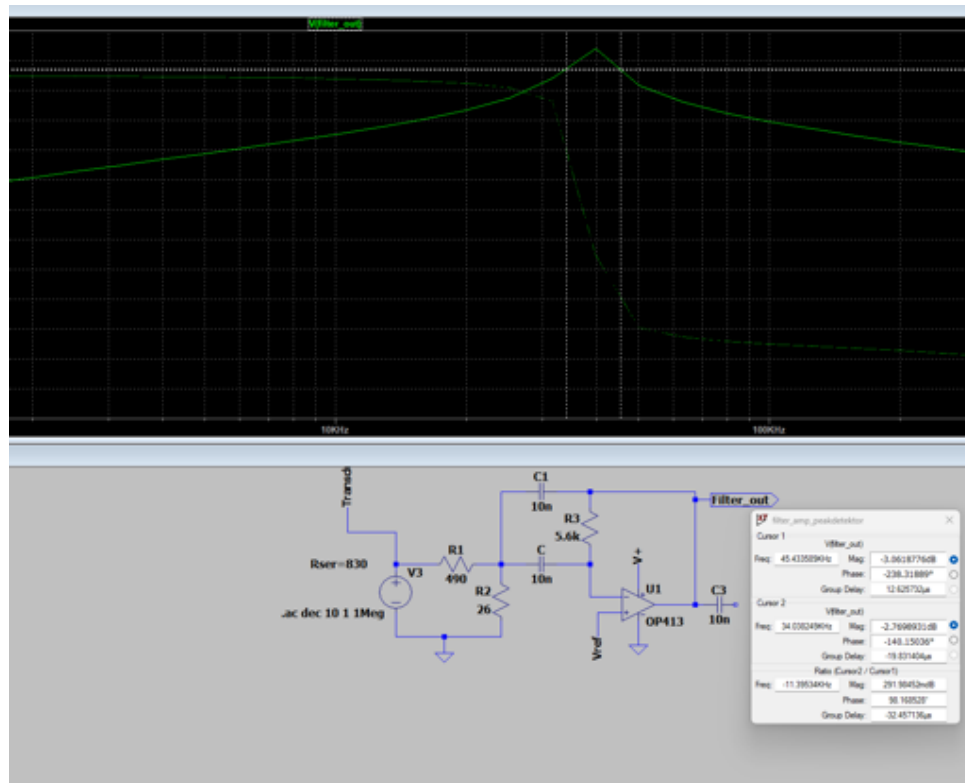


Figure 22: Cut Off Region

Reflection As showed in Figure 21, and 22 the center $f_c=41\text{kHz}$, $f_h=45\text{kHz}$ $f_l=35\text{kHz}$ this results still acceptable for our case because the 43kHz still close to the center frequency. The reason of this results can be:

Simulation Model Accuracy: the Lt-Spice is as a virtual lab. Just like real components, the virtual ones aren't perfect and might not always behave as expected, leading to some surprising results.

Component Tolerances in Simulation: Just as real components vary slightly, simulations can include these variations too. Programming these tolerances helps mimic real-world behavior but can also lead to slight discrepancies from the ideal

6.5.2 Inverting amplifier

Initially, the signal encounters capacitor $C3$, serving primarily as a DC blocker. Concurrently, in conjunction with resistor $R4$, it forms a high-pass filter. The cutoff frequency (f_c) for this filter is calculated using $f_c = \frac{1}{2\pi RC}$, yielding $f_c = \frac{1}{2\pi \cdot 1\text{k}\Omega \cdot 10\text{nF}} = 15.9\text{kHz}$. Moreover, capacitor $C3$ experiences an AC voltage drop due to the 43kHz frequency passing through it, with its reactance (X_{c3}) determined by $X_{c3} = \frac{1}{2\pi fC} = \frac{1}{2\pi \cdot 43\text{kHz} \cdot 10\text{nF}} = 370\Omega$.

The signal then proceeds to $U1.2$, which functions as an inverting amplifier. This amplifier, through the use of resistors $R5$ and $R4$, is capable of amplifying the signal 50 times. The amplification factor (A_u) is given by $A_u = -\frac{R5}{R4} = -\frac{50\text{k}\Omega}{1\text{k}\Omega} = 50$. Consequently, the input signal at $U2$ is amplified 50x.

To accommodate for the lack of negative supply, the non-inverting pin is tied to half of the battery voltage, establishing a virtual ground.

6.5.3 Signal processing

Arduino family typically features a 10-bit ADC. It operates at a maximum ADC clock speed of about 200 kHz. The optimal ADC conversion time is around 100 microseconds per sample, leading to a maximum sample rate of 10 kHz under ideal conditions $\text{samplerate} = \frac{1}{100\mu\text{s}} = 10\text{kHz}$. Inadequate Sampling Rate: The 10 kHz maximum sampling rate of the Arduino is significantly lower than the 43 kHz signal frequency. To effectively capture the signal without aliasing,

according to the Nyquist theorem, you need a sampling rate that is at least twice the highest frequency component of the signal — in this case, ideally above 86 kHz.

Real-World Performance: The theoretical maximum often doesn't account for additional time required for the program execution, data handling, and other peripheral activities in the Arduino . These factors further reduce the effective sampling rate in practical applications, therefore we found a different solution to process received signal. It is by the use of peak detector.

6.5.4 Peak-Detector Circuit

Peak detector is like a clever shortcut. Instead of trying to catch every tiny detail of the 43 kHz signal (which the Arduino can't keep up with), the peak detector just grabs the highest points of the signal. Here's what it does for you:

Simplifies the Signal: It breaks down the complicated signal into just the peaks. This way, we don't need to sample as often to get the gist of what your signal looks like.

So, while the Arduino alone can't catch every detail of a 43 kHz signal, with a peak detector's help, it can grab enough info to be useful for many applications.

Components and Functionality:

- **OPA4134:** Configured as a buffer or voltage follower to provide high input impedance and low output impedance.
- **Diode (D2):** Facilitates charge accumulation on capacitor $C5$ while preventing discharge back through the op-amp when the input voltage falls below the peak value. It allows conduction when the input voltage surpasses the voltage on $C5$, thereby charging $C5$ to the peak input signal value."See figure [20](#)"
- **Capacitor (C5):** Stores the peak voltage. The capacitance value determines the response time of the peak detection circuit.
- **MOSFET (Q1):** is to discharge $C5$ upon receiving a signal from the Arduino. When activated, $C5$ discharges through $R8$.

Operational Analysis

The voltage across capacitor $C4$ during charging is described by:

$$V_C = V_{\text{initial}} \left(1 - e^{-\frac{t}{RC}} \right) \quad (8)$$

The charge time for $C4$ is given by:

$$C5 \text{ charge time} = 5 \cdot (R_{\text{opa_out_impedance}} + R_{\text{frw_diode}}) \cdot C \quad (9)$$

For a specific case:

$$t = -RC \cdot \ln \left(1 - \frac{V_{\text{initial}}}{V_{\text{final}}} \right) \quad (10)$$

$$t = -0.1\Omega \cdot 220\text{pF} \cdot \ln \left(1 - \frac{0.5}{5} \right) = 1.\text{ps}$$

The calculated charge time:

$$= 5 \cdot (0.1 \cdot 220\text{p}) = 110\text{ps}$$

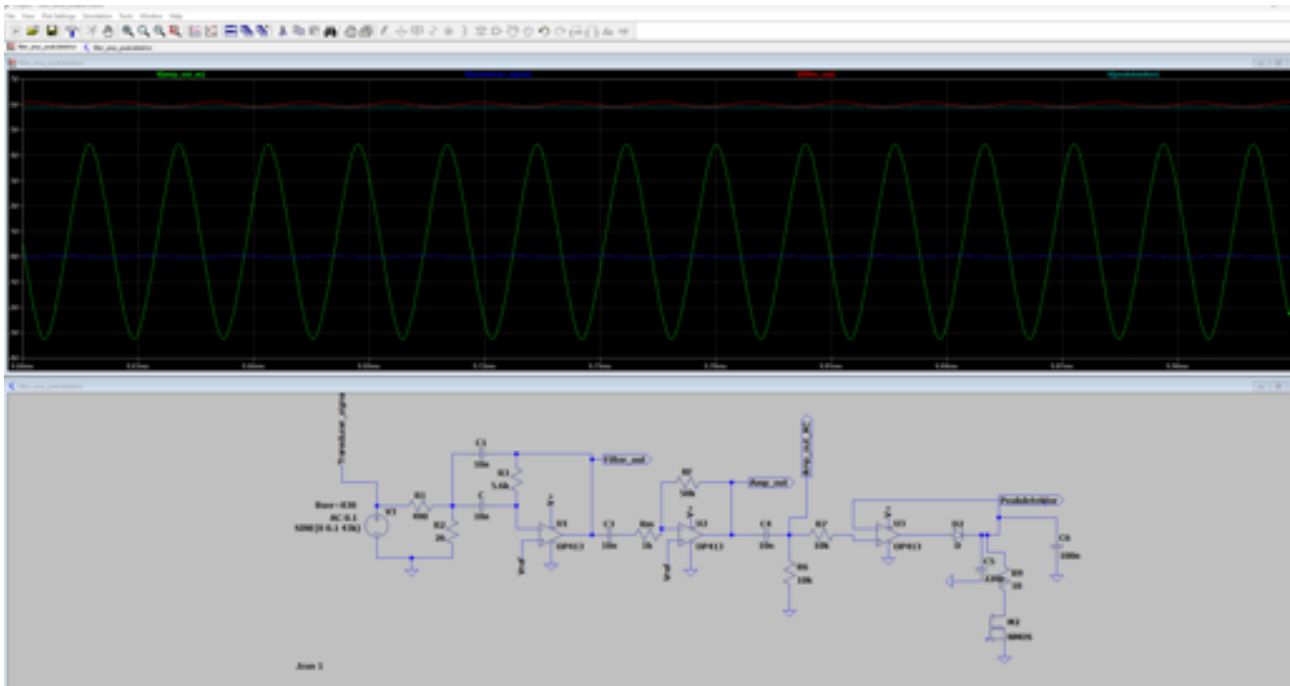
The discharge time considering resistances $R8$ and R_{ds} :

$$\text{Discharge time} = 5(R_9 + R_{ds}) \quad (11)$$

For a specific discharge case:

$$t = -RC \cdot \ln\left(\frac{V_{\text{final}}}{V_{\text{initial}}}\right) \quad (12)$$

$$t = -(10.016\Omega \cdot 220\text{pF}) \cdot \ln\left(\frac{0.5}{6}\right) = 5.48\text{ns}$$



The TX circuit consist of MFB Band Pass Filter, and Common Emitter Amplifier.

6.6.1 MFB Band Pass Filter:

The MFB filter at the Tx circuit is a copy of the one at the RX circuit. The signal is 43kHz square wave generated by the Arduino, the signal enters MFB band pass filter where the harmonies getting filtered out, and the 43kHz will pass in a wave form

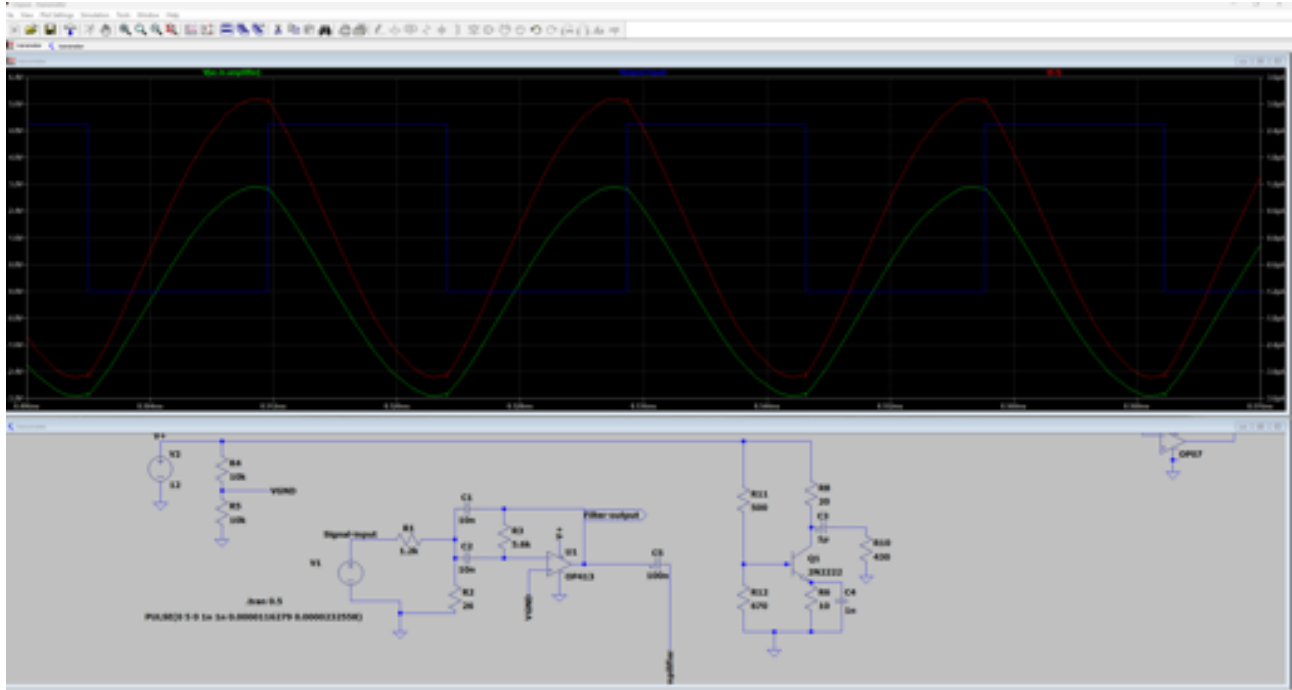


Figure 25: TX Band Pass Filter

6.6.2 Common Emitter Amplifier:

The main purpose of the Common Emitter Amplifier, is to amplify the voltage and the current of the signal before driving it to the Transducer. R17, and R18 making the base voltage to run Q2. the relationship between R19, and R20 define the voltage amplifying.

$$Z_{in} = R_{17} \parallel R_{18} \parallel h_{ie} \quad (13)$$

$$h_{ie} = r_e \cdot h_{fe} \quad (14)$$

$$r_e = \frac{25mV}{I_E} \quad (15)$$

$$A_u = 40 \cdot I_E \cdot (R_C \parallel R_L \parallel \frac{1}{h_{oe}}) \quad (16)$$

$$I_C = I_B \cdot h_{fe} \quad (17)$$

Understanding the relationship between these variables and power output is essential for adjusting the circuit to achieve optimal signal transmission. The group encountered difficulties in determining the efficiency and sensitivity of the transducer, which led to challenges in calculating the components of the common emitter amplifier. But equations [13](#), [14](#), [15](#), [16](#), [17](#) are the formulas needed to achieve the correct power needed to transit the signal X meters.

6.7 Main PCB

During the project's testing phase, we created our own PCB using a Veroboard. Additionally, we constructed a prototype of the main PCB for demonstration purposes, as illustrated in figures [26](#) and [27](#)

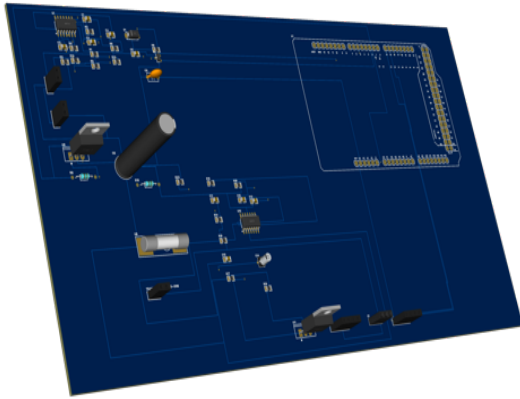


Figure 26: Main PCB 3D model

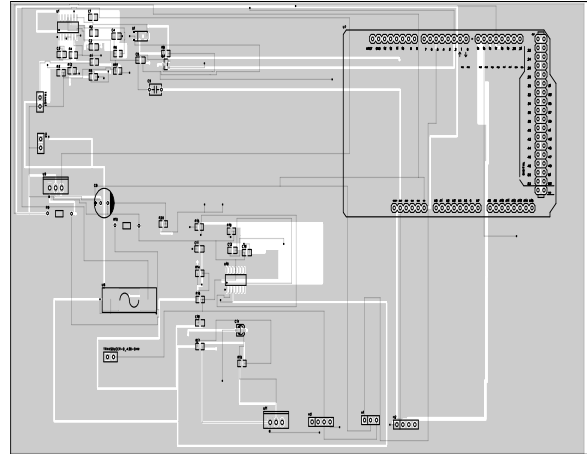


Figure 27: Main PCB schematics

7 Programming and Algorithms

7.1 GPS Module

7.1.1 Introduction to GPS

The GPS module is responsible for acquiring latitude and longitude coordinates for the boat and the bracelet. The module used in this project was the NEO-6M GPS module. This module has a very convenient configuration for usage as it only requires four pins. The communication protocol used in the module is UART.

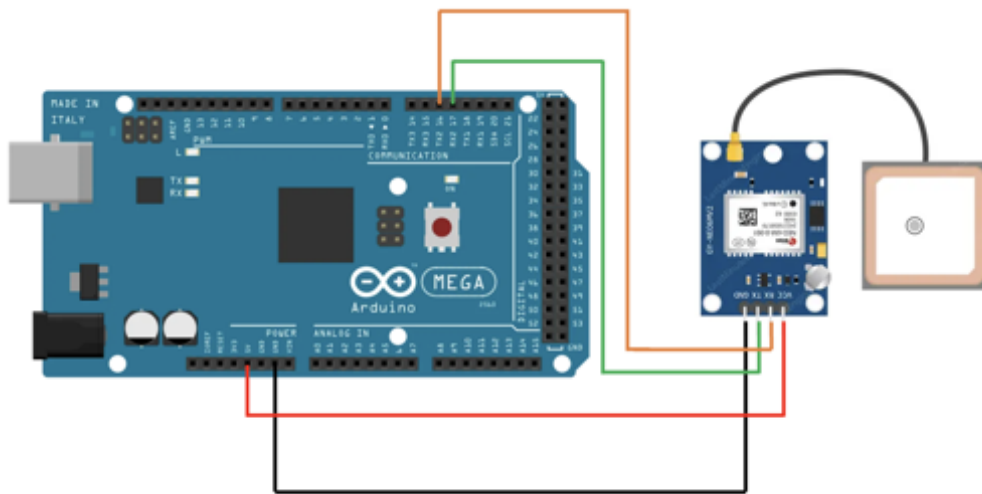


Figure 28: Typical wire connections between an Arduino (in this case, the Mega) and the NEO-6M GPS module.

Since the operations of the boat are controlled by an Arduino Mega, connections similar to that shown in figure 28 is suitable. The bracelet uses an Arduino Nano which only has one UART protocol set (i.e one RX and one TX). To allow UART communication with other sensors, only the RX pin in the Nano can be used for this GPS so that the TX can be used with the RF module for transmitting data to the boat.

The NEO-6M module includes a signal-receiving LED which blinks at a frequency of 1 Hz when it is receiving signals from satellites. Even in overcasting, rainy or windy conditions, signals were still observed in the module. When testing indoors under aluminium pipes and interfering electrical circuits, no signals were observed.

7.1.2 Parsing of NMEA Sentences

The NEO-6M transmits data via its TX pin as a sequence of characters which build up an NMEA (National Marine Electronics Association) sentence. Typically, this consists of several values such as latitude, longitude, speed and time which are encoded into certain lines/strings. Examples of these lines are 4:

- GGA: Time, position and fixed type data
- ZDA: PPS timing message
- RMC: Time, date, position, course and speed data
- MSS: Signal-to-noise ratio, signal strength and frequency

The most popular and convenient line is the RMC. This gives the latitude, longitude and speed of the GPS module. Figure 29 shows the serial monitor result after reading from the RX pin of the Arduino.

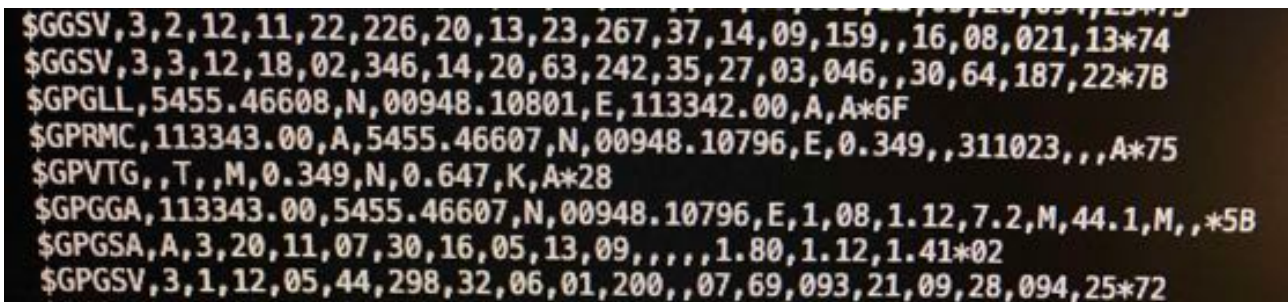


Figure 29: Example of an NMEA Sentence outputted to the serial monitor.

Since each character is sent individually from the NEO-6M through UART, a string has to be built by adding the character to the end of the string. This string represents each line of the NMEA sentence. Therefore, upon receiving a new line (i.e \$GGSV or \$GPRMC), the string must be cleared and then used again to store the next line. The process for getting the latitude is described below.

1. Check if the character received is "\n". If so, this means that the previous string has finished building and a new one will be built.
2. From the previous built string, check if it includes the string "GPR". If so, parse the string to get the latitude and longitude.
3. Clear the string.

The parsing is done using an external C library with a header file, `minmea.h` [5]. The following code demonstrates the procedure described above by implementing an RX interrupt function and saving the data to global volatile variables.

```
ISR(USART3_RX_vect){
    // Parse data
    received_data = UDR3;
    if (received_data == '\n') {
        // Parse formed line
        if(strstr(received_string, "GPR")){
            struct minmea_sentence_rmc frame;
            if (minmea_parse_rmc(&frame, received_string)) {
                latitude = minmea_tocoord(&frame.latitude);
                longitude = minmea_tocoord(&frame.longitude);
                speed = minmea_tofloat(&frame.speed);
                boatCoordinateReceived = 1;
            }
            else {
                printf("Error! Not parsed correctly!");
            }
        }

        // Store a new line
        received_string[string_index] = '\0'; // Null-terminate the string
        string_index = 0; // Reset the string index
    } else {
        received_string[string_index++] = received_data;
    }
}
```

The interrupt utilises the `minmea.h` library to parse a GPRMC string and then save the data to `minmea_sentence_rmc`.

7.1.3 UART Setup

The BAUD-rate of a communication protocol defines the number of signal units sent per second. The NEO-6M can actually operate with fast BAUD-rates. The following code demonstrates setting up the interrupts on RX and TX in the Arduino at a BAUD-rate of 9600.

```
#define BAUDRATE 9600
#define BAUD_PRESCALER (((F_CPU / (BAUDRATE * 16UL))) - 1)

void USART_init(void){
    UBRR3H = (uint8_t)(BAUD_PRESCALER>>8);
    UBRR3L = (uint8_t)(BAUD_PRESCALER);
    UCSR3B = (1<<RXEN3)|(1<<TXEN3);
    UCSR3C = ((1<<UCSZ30)|(1<<UCSZ31));
}
```

7.2 RF Module

7.2.1 Introduction to RF

The RF (radio frequency) module enables communication between two devices. In this project, RF communication is used to send the coordinates of the bracelet from it to the boat. Therefore, a transmitter and a receiver should be used. For convenience, a transceiver is often used as it supports both transmitting and receiving and so has an RX and TX pin.

Several modules were experimented with such as the NRF24L01, which uses SPI communication, and the HC12, which uses UART communication. The NRF24L01 caused complications in circuitry and programming due to the relative complexity of SPI compared to UART and the fact that it has 8 pins. Unlike this, the HC12 module only requires 4 pins to function (VCC, GND, RX and TX). It has a suitable range for this project with a maximum range of approximately 1 km.

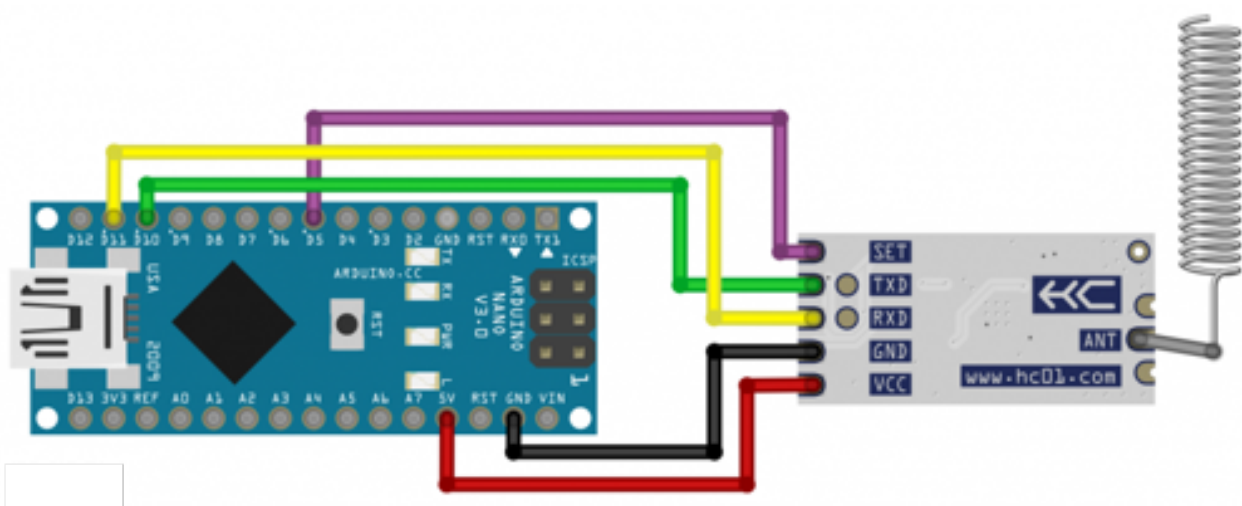


Figure 30: Typical wire connections between an Arduino (in this case, the Nano) and the HC12 RF module. [6]

Similar to the NEO-6M module in the bracelet, only the TX pin in the Arduino Nano will be connected to the HC12 to allow both the GPS and the RF modules to function in one UART. The SET pin on the HC12 allows further settings to be controlled. This pin was not used in this project.

7.2.2 GPS Precision for RF Communication

The HC12 only supports the transmission of char value types which are 8 bits. However, the values for latitude and longitude go up to several decimal places and are stored in float variables. Around the campus (SDU, Sønderborg), the latitude and longitude follow approximately this structure:

- Latitude: 54.912569
- Longitude: 9.780349

At 100 km east of the campus, the longitude value changes to two digits before the decimal point. For simplicity, it is assumed that the latitude always stays at *two digits before the decimal point* and the longitude at *one digit before the decimal point*. Also, a coordinate precision of six decimal places is kept. This ensures an uncertainty in the eastward and northward directions of just $\approx 10\text{cm}$ as shown here:

$$\Delta_{\text{position}} = \frac{\text{Precision}}{360^\circ} \times 2\pi R \quad (18)$$

where *Precision* is the smallest digit precision and *R* is the radius of the Earth. Then,

$$\Delta_{\text{position}} = \frac{0.000001^\circ}{360^\circ} \times 2\pi \cdot 6,371,000\text{m} = 0.11\text{m}$$

Since this uncertainty is negligible, the total inaccuracy in the location is that of the GPS module, 2.5 m.

7.2.3 Transmission: Cycle and Typecasting

The process of sending the coordinates was devised and is represented below:

1. Send "A" - Latitude
2. Send each digit of the latitude sequentially
3. Send "O" - Longitude
4. Send each digit of the longitude sequentially
5. Send "F" - Finish
6. Repeat

The letters, "A", "O" and "F" are needed to specify the successive sequence of data. To transmit data via the Arduino's TX, an unsigned char is written to the UDRn register. So, each specification letter and digit of the coordinates must be translated to char before sending. This is done with the help of an ASCII table.

By deduction from figure [31](#), the offset between a char representing a digit (0-9) and the corresponding dec is 48. The few lines of code below demonstrates this type conversion ready for transmission.

```
int main(void){
    ...
    char digit = latDigits[i] + 48;
    USART_Transmit(digit);
    ...
}

void USART_Transmit(unsigned char data){
    while (!(UCSR0A & (1 << UDRE0)));
    UDR0 = data;
}
```

Here, `(latDigits[i] + 48)` is of type `int`. For example, if `latDigits[i]` is integer 5, then `(latDigits[i] + 48)` equals `5 + 48` which is 53. The typecasted assignment to the `char` variable called `digit` will convert the 53 to a `char`. According to figure 31, this 53 in `char` is 5 which is what `latDigits[i]` is equal to in integer form. Finally, a `char` value of 5 is transmitted.

Dec	Hex	Oct	Chr	Dec	Hex	Oct	HTML	Chr	Dec	Hex	Oct	HTML	Chr	Dec	Hex	Oct	HTML	Chr
0	0	000	NULL	32	20	040	 	Space	64	40	100	@	@	96	60	140	`	~
1	1	001	Start of Header	33	21	041	!	!	65	41	101	A	A	97	61	141	a	a
2	2	002	Start of Text	34	22	042	"	"	66	42	102	B	B	98	62	142	b	b
3	3	003	End of Text	35	23	043	#	#	67	43	103	C	C	99	63	143	c	c
4	4	004	End of Transmission	36	24	044	$	\$	68	44	104	D	D	100	64	144	d	d
5	5	005	Enquiry	37	25	045	%	%	69	45	105	E	E	101	65	145	e	e
6	6	006	Acknowledgment	38	26	046	&	&	70	46	106	F	F	102	66	146	f	f
7	7	007	Bell	39	27	047	'	'	71	47	107	G	G	103	67	147	g	g
8	8	010	Backspace	40	28	050	(	(	72	48	110	H	H	104	68	150	h	h
9	9	011	Horizontal Tab	41	29	051	)	)	73	49	111	I	I	105	69	151	i	i
10	A	012	Line feed	42	2A	052	*	*	74	4A	112	J	J	106	6A	152	j	j
11	B	013	Vertical Tab	43	2B	053	+	+	75	4B	113	K	K	107	6B	153	k	k
12	C	014	Form feed	44	2C	054	,	,	76	4C	114	L	L	108	6C	154	l	l
13	D	015	Carriage return	45	2D	055	-	-	77	4D	115	M	M	109	6D	155	m	m
14	E	016	Shift Out	46	2E	056	.	.	78	4E	116	N	N	110	6E	156	n	n
15	F	017	Shift In	47	2F	057	/	/	79	4F	117	O	O	111	6F	157	o	o
16	10	020	Data Link Escape	48	30	060	0	0	80	50	120	P	P	112	70	160	p	p
17	11	021	Device Control 1	49	31	061	1	1	81	51	121	Q	Q	113	71	161	q	q
18	12	022	Device Control 2	50	32	062	2	2	82	52	122	R	R	114	72	162	r	r
19	13	023	Device Control 3	51	33	063	3	3	83	53	123	S	S	115	73	163	s	s
20	14	024	Device Control 4	52	34	064	4	4	84	54	124	T	T	116	74	164	t	t
21	15	025	Negative Ack.	53	35	065	5	5	85	55	125	U	U	117	75	165	u	u
22	16	026	Synchronous idle	54	36	066	6	6	86	56	126	V	V	118	76	166	v	v
23	17	027	End of Trans. Block	55	37	067	7	7	87	57	127	W	W	119	77	167	w	w
24	18	030	Cancel	56	38	070	8	8	88	58	130	X	X	120	78	170	x	x
25	19	031	End of Medium	57	39	071	9	9	89	59	131	Y	Y	121	79	171	y	y
26	1A	032	Substitute	58	3A	072	:	:	90	5A	132	Z	Z	122	7A	172	z	z
27	1B	033	Escape	59	3B	073	;	:	91	5B	133	[	[	123	7B	173	{	{
28	1C	034	File Separator	60	3C	074	<	<	92	5C	134	\	\	124	7C	174	|	
29	1D	035	Group Separator	61	3D	075	=	=	93	5D	135	]	]	125	7D	175	}	}
30	1E	036	Record Separator	62	3E	076	>	>	94	5E	136	^	^	126	7E	176	~	~
31	1F	037	Unit Separator	63	3F	077	?	?	95	5F	137	_	_	127	7F	177		Del

asciicharstable.com

Figure 31: An ASCII table. 7

To summarise this method, the aim was to store an `int` digit into a `char` variable without changing its value. In the above example, the digit value, `int` 5, was stored as a `char` 5 for transmission. A direct typecast without the 48 offset would mean that the transmission value would be Enquiry according to figure 31.

7.2.4 Transmission: Extracting Digits

We have seen that the latitude values have 8 digits and the longitude values have 7 digits. This is represented here,

```
int latDigits[8];
int longDigits[7];
```

The goal is to extract each digit from the `latitude` and `longitude` float variables and store them into `latDigits` and `longDigits` respectively. This is shown in the code below. The idea is to shift the lower-base digits left and then use a modulus operation on the new float.

Operations for Latitude: (8 digits)

```
// Get latitude digits
latDigits[0] = ((long int)(latitude) % 100) / 10;
latDigits[1] = ((long int)(latitude) % 10) / 1;
latDigits[2] = ((long int)(latitude * 10)) % 10;
latDigits[3] = ((long int)(latitude * 100)) % 10;
latDigits[4] = ((long int)(latitude * 1000)) % 10;
latDigits[5] = ((long int)(latitude * 10000)) % 10;
latDigits[6] = ((long int)(latitude * 100000)) % 10;
latDigits[7] = ((long int)(latitude * 1000000)) % 10;
```


Operations for Longitude: (7 digits)

```
// Get longitude digits
longDigits[0] = ((long int)(longitude) % 10) / 1;
longDigits[1] = ((long int)(longitude * 10)) % 10;
longDigits[2] = ((long int)(longitude * 100)) % 10;
longDigits[3] = ((long int)(longitude * 1000)) % 10;
longDigits[4] = ((long int)(longitude * 10000)) % 10;
longDigits[5] = ((long int)(longitude * 100000)) % 10;
longDigits[6] = ((long int)(longitude * 1000000)) % 10;
```

Now that each digit is acquired, the transmission cycle as described in section [7.2.3](#) can be implemented as shown below. First the latitude is sent, then the longitude and finally the message for finishing sending the coordinates.

```
// Send latitude
USART_Transmit(65); // 65 is 'A': Latitude
_delay_ms(10);
for(int i = 0; i < 8; i++){
    char digit = latDigits[i] + 48;
    USART_Transmit(digit);
    _delay_ms(10);
}
// Send longitude
USART_Transmit(79); // 65 is 'O': Longitude
_delay_ms(10);
for(int i = 0; i < 7; i++){
    char digit = longDigits[i] + 48;
    USART_Transmit(digit);
    _delay_ms(10);
}
// Send finished
USART_Transmit(70); // 70 is 'F': Finish
```

The procedure repeats every time the received latitude and longitude update from the RX interrupt for the GPS module.

7.2.5 Receiving: Rebuilding Coordinates

On the receiving side (boat), an interrupt is set up for the RX2 pin connected to the RF module. The following code, written inside the interrupt, receives the digits and appends them to the latDigits and longDigits of the boat.

```
ISR(USART2_RX_vect){
    received_data = UDR2;
    int digit = received_data - 48;
    ...
    if(latOrLong == 17){ // latitude
        latDigits[digitNumber] = digit;
        digitNumber++;
    } else if(latOrLong == 31){ // longitude
        longDigits[digitNumber] = digit;
        digitNumber++;
    }
    ...
}
```

Here, digitNumber keeps track of the digit place in the floating structure of the coordinates. It is reset to 0 when 'F' is received which corresponds to fully transmitted coordinates. Notice that int digit subtracts 48 from received_data to account for the aforementioned offset from char int.

Now that the digits are obtained, the following function is executed when the message 'F' is received. This function in particular builds the latitude float from the digits. The computed value is stored into a variable denoting the destination latitude. The same procedure occurs for the longitude.

```
float destLatitude = 0;

ISR(USART2_RX_vect){
    ...
    if(digit == 22){ // finished
        destLatitude = computeLatitude();
        ...
    }
}

float computeLatitude(){
    float calculatedLat = (
        latDigits[0] * 10 +
        latDigits[1] * 1 +
        latDigits[2] * 0.1 +
        latDigits[3] * 0.01 +
        latDigits[4] * 0.001 +
        latDigits[5] * 0.0001 +
        latDigits[6] * 0.00001 +
        latDigits[7] * 0.000001
    );
    return calculatedLat;
}
```

Despite adding long data types, the values of the calculated latitudes and longitudes actually varied by ± 0.000003 . This occurs due to the 6-7 digit precision of floating values in the Arduino. Nevertheless, according to equation 18, the uncertainty in this phenomenon is approximately 35 cm which is not relatively significant.

7.2.6 Transmitting: Activating Starting Protocols

The boat will only start once the button on the bracelet has been clicked. To implement this functionality, a simple circuit with the Arduino Nano and a button was constructed. The button used was an SPNO push switch which is 'normally open'.

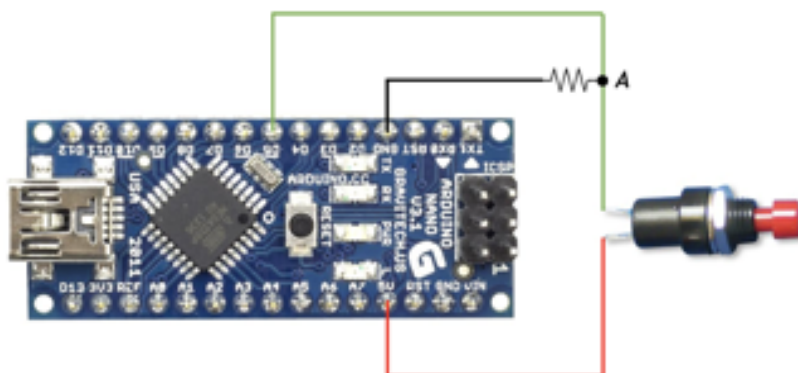


Figure 32: Wire connections between an Arduino (in this case, the Nano) and a SPNO button circuit.

The connections shown in figure 32 mean that when the button is up, the D5 pin reads low (0 V). D5 reads the digital value at node A. When the button is up, the reading at D5 is low since no current flows across the button

due to its 'normally open' nature. An if statement which checks when the button is pressed can be implemented as shown below. The starting message, 'S', is also transmitted.

```
while(1){
    if((PIND & (1 << DD5)) == (1 << DD5) && startRescueTransmission == 0){
        USART_Transmit(83); // 83 is "S": Start
        startRescueTransmission = 1;
    }
    ...
}
```

PIND has a normal value of 0x00. The only connection to the PORTD register on the bracelet is of the button. Therefore, a high reading on D5 can be checked by applying an AND operator (&) with (1 << DD5). The startRescueTransmission is a flag which is set when the button is first clicked. This keeps track of the initial press and disables the user from sending starting protocols to the boat repeatedly.

7.3 Digital Compass

7.3.1 Introduction to Digital Compass

The addition of the digital compass/magnetometer gives the boat information of heading. The heading is the angle from North that the boat is facing. Based on the destination heading, a mathematical controller can be implemented to direct the boat from its original heading to its destination heading. Without the magnetometer, the only way to acquiring a heading (although quite inaccurately) is by moving the boat along a random direction and calculating the heading based on the change in GPS coordinates.

The magnetometer used is the HMC5883L which uses I2C communication. The connections are shown below.

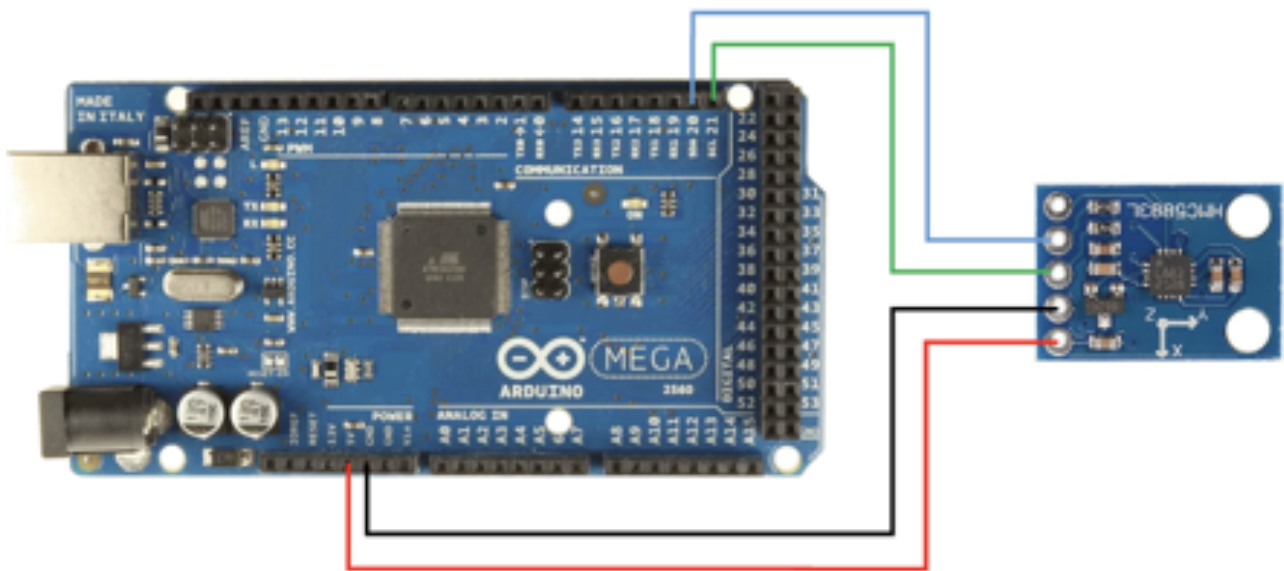


Figure 33: Typical wire connections between an Arduino (in this case, the Mega) and the HMC5883L Compass module.

The HMC5883L has three axes which each measure the magnetic field strength along their own axis. Also, the magnetic field direction of the Earth is never parallel to the ground, except for in the equator, but is rather tilted and coming out/going into it. For this reason, if the magnetometer is kept level with the ground, the z-axis of the magnetometer should still read a value.

For sensors which use I2C communication, the sensor be initialized. The data values from the sensor are stored in specific registers which can be found by referring to the datasheet of the sensor. For the HMC5883L, the following functions initializes the sensor and returns the x axis reading respectively [8].

```
void Magneto_init(){ /* Magneto initialize function */
    i2c_start(0x3C); /* Start and write SLA+W */
    i2c_write(0x00); /* Write memory location address */
    i2c_write(0x70); /* Configure register A as 8—average, 15 Hz default, normal measurement */
    i2c_write(0xA0); /* Configure register B for gain */
    i2c_write(0x00); /* Configure continuous measurement mode in mode register */
    i2c_stop(); /* Stop I2C */
}
```

```
int Magneto_GetX(){
    int x, y, z;
    double Heading;
    i2c_start_wait(0x3C); /* Start and wait for acknowledgment */
    i2c_write(0x03); /* Write memory location address */
    i2c_rep_start(0x3D); /* Generate repeat start condition with SLA+R */
    /* Read 16 bit x,y,z value (2's complement form) */
    x = (((int)i2c_readAck()<<8) | (int)i2c_readAck());
    z = (((int)i2c_readAck()<<8) | (int)i2c_readAck());
    y = (((int)i2c_readAck()<<8) | (int)i2c_readAck());
    i2c_read(0); // Must to a NAK on the last read
    i2c_stop(); /* Stop I2C */
    return x;
}
```

7.3.2 Calibration and Heading Calculation

The magnetometer is very sensitive to magnetic fields. Any electronic device which possesses a magnetic field will to some extent, add an offset to each axis reading of the magnetometer. For this reason, it must be calibrated every time it is in a new environment. Types of offsets which occur due to surrounding magnetic fields are [9]:

- Hard-iron: Offsets the readings
- Soft-iron: Stretches the readings

The hard-iron offsets are most predominant and occur due to objects possessing their own magnetic field. The soft-iron distortion occurs mainly due to wires and objects with iron and nickel. A more visual method to show these offsets are in a 2D plot. The following code continuously prints out the x, y and z readings in a CSV format.

```
int main(void){
    ...
    while(1){
        int x = Magneto_GetX();
        int y = Magneto_GetY();
        int z = Magneto_GetZ();
        printf("%d, %d, %d\n", x, y, z);
        _delay_ms(50);
    }
}
```

By rotating the magnetometer on its z-axis while keeping it level, a set of points is acquired which should produce a perfect circle. Figure [34] shows the result of this test. The orange circle is the raw plot while the blue shows the readings calibrated manually. Here, the hard-iron offset is clearly visible as it shifts the circle away from the origin.

This almost-perfect circle proves that the heading can easily be calculated using basic trigonometry,

$$\text{Heading, } \phi = \arctan\left(\frac{y}{x}\right) \quad (19)$$

For example, at the points where the calibrated circle (blue) touches the x-axis, the y reading is 0 since it is exactly perpendicular to the Earth's magnetic field. Here, the x axis is aligned with Earth's magnetic field.

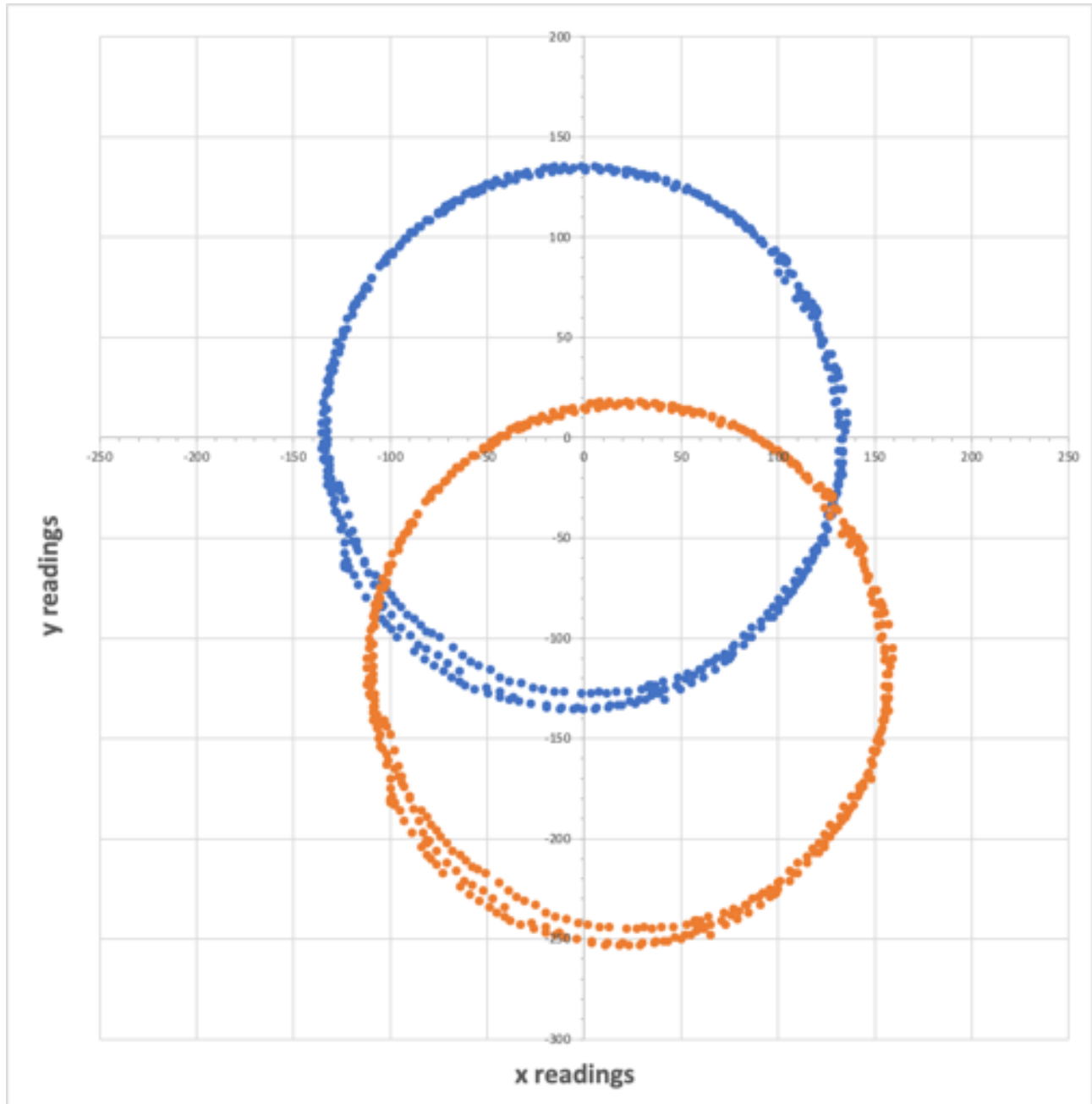


Figure 34: A plot of the x and y readings when rotating the magnetometer on its z axis while keeping it level. The orange circle represents the raw readings and the blue circle represents the calibrated readings.

The plot in figure 34 was constructed by manually rotating the magnetometer. However, this process needs to be done automatically. During the rotation, the magnetic field strengths need to be recorded and then calibrated automatically. This process requires the usage of a servo motor.

For this project, the MG996R servo motor was used for automatic calibration. This servo motor has 180 ° of free

rotation. So, instead of a full circle, only an arc will be seen. For this automatic calibration, the magnetometer will need to be elevated to around 15 cm above the servo motor as it is a source of changing magnetic fields. An initial setup is shown below in figure 35. Later, a more robust and modular design was made as shown in figure 36.



Figure 35: Initial setup for automatic compass calibration

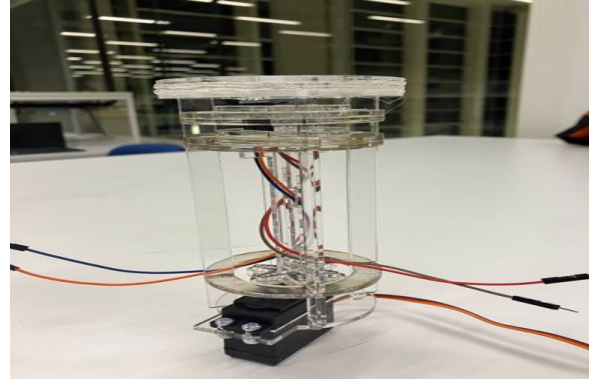


Figure 36: Final setup for automatic compass calibration.

Rotating the servo motor is achieved through sending PWM signals to its signal pin. The PWM is initialized using the following code.

```
int main(void){
    ...
    DDRE |= (1<<PE3); // Set PB5 as output
    TCCR3A |= (1<<COM3A1) | (1<<WGM31); // Fast PWM, non-inverting mode
    TCCR3B |= (1<<WGM33) | (1<<WGM32) | (1<<CS31); // Fast PWM, prescaler = 8
    ICR3 = 39999; //20ms PWM period
    ...
}
```

The PWM can be varied by changing the value of OCR3A. The PWM values of OCR3A which correspond to the starting (0 °), neutral (90 °) and ending (180 °) positions are defined globally:

```
#define START 999
#define NEUTRAL 2999
#define END 4999
```

A different program was written which outputs the x, y and z readings from the magnetometer while simultaneously rotating the servo by increments of 5 °. The orange graph in figure 37 shows this plot. To obtain the calibrated plot automatically, surprisingly, only two points are needed. To find the offset, the center of curvature of the arc needs to be determined.

Suppose that the following variables are defined:

x_0 : Reading of x at the starting position of the servo

y_0 : Reading of y at the starting position of the servo

x_1 : Reading of x at the ending position of the servo

y_1 : Reading of y at the ending position of the servo

Then, the center of curvature of the arc is at (assuming the arc is a circle),

$$x_c = x_0 + \frac{x_1 - x_0}{2}$$

$$y_c = y_0 + \frac{y_1 - y_0}{2}$$

These formulas describe the number exactly between the starting and ending x and y values. The offset shifts the arc towards the origin. So, the offsets are just the centers of curvature multiplied by -1 .

$$x_{off} = -\left(x_0 + \frac{x_1 - x_0}{2}\right) \quad (20)$$

$$y_{off} = -\left(y_0 + \frac{y_1 - y_0}{2}\right) \quad (21)$$

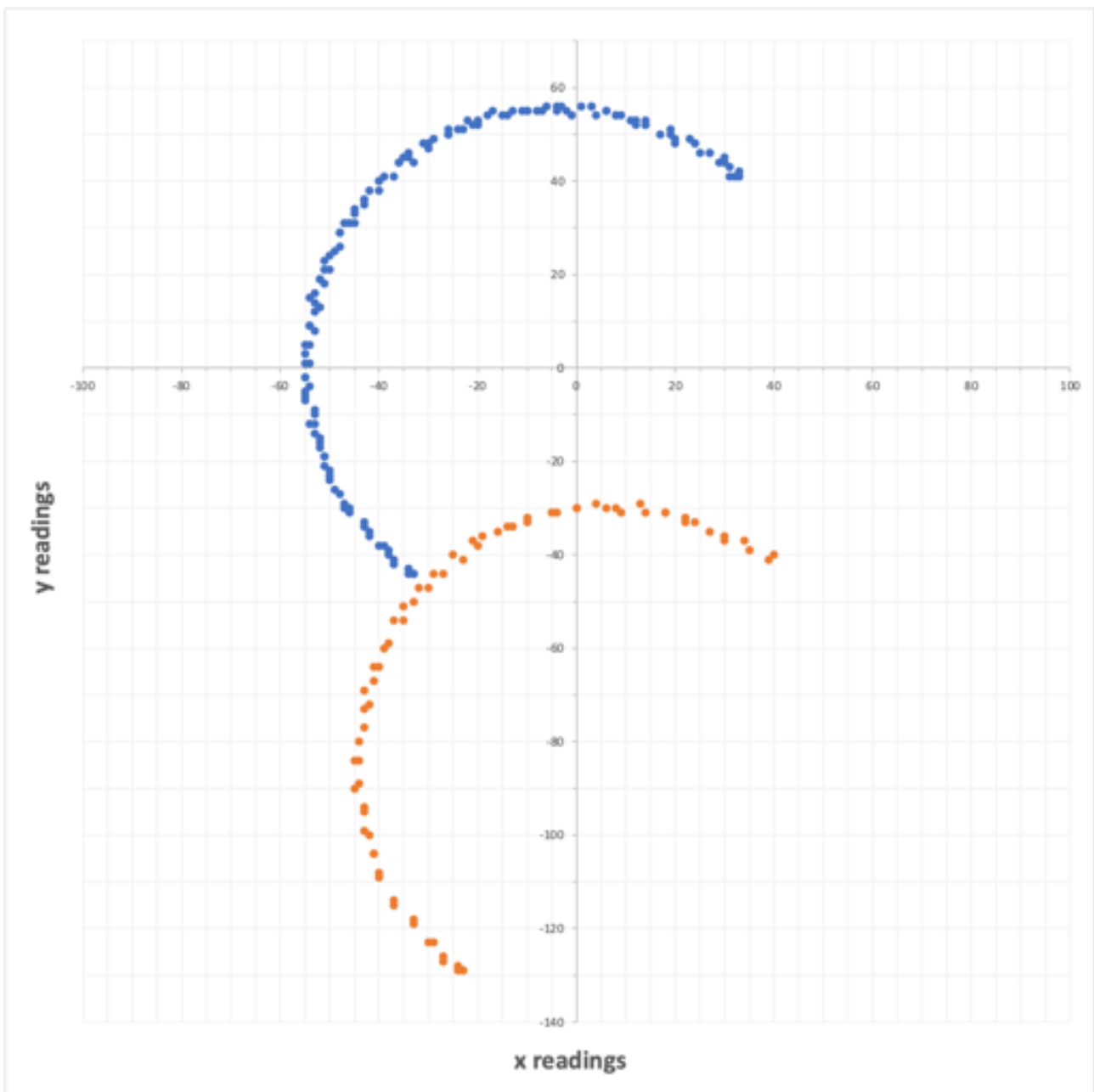


Figure 37: A plot of the x and y readings from an automatic calibration.

The automatic calibration brings several advantages such as smooth readings with minimal vibration, increased calibration time, increased accuracy in calibration etc.

So, to calibrate the compass automatically, the program should follow these steps:

1. Set motor position to start
2. Read x and y
3. Set motor position to end
4. Read x and y
5. Set motor position to start (reset)
6. Use equations [20](#) and [21](#) to calculate offsets.

The following function implements this algorithm in C.

```
void Calibrate_Magnetometer(){
    int startValueX, startValueY, endValueX, endValueY;

    OCR3A = START; // Set position to 0 degrees
    _delay_ms(1000);

    startValueX = Magneto_GetX();
    startValueY = Magneto_GetY();
    _delay_ms(500);
    OCR3A = END;
    _delay_ms(1000);

    endValueX = Magneto_GetX();
    endValueY = Magneto_GetY();
    _delay_ms(500);
    OCR3A = START;
    _delay_ms(1000);

    int deltaX = endValueX - startValueX;
    int deltaY = endValueY - startValueY;
    offsetX = -1 * (startValueX + (deltaX / 2));
    offsetY = -1 * (startValueY + (deltaY / 2));
}
```

7.4 Magnetometer Tilt Compensation

7.4.1 Problems with Tilt

The magnetometer is calibrated once on a flat surface approximately parallel to the ground. This calibration results in an accurate heading which can be used. However, we observed that tilting the magnetometer, even by 5-10 °, causes the heading to vary by over 50 °. This problem will be very apparent when the boat drives since waves, turning and other maneuvers will cause the boat to tilt. The algorithm devised to tackle these types of problems is known as Tilt Compensation.

This technique takes the magnetometer x, y and z readings and projects it back to the original plane where the magnetometer was calibrated. A visual representation of this is shown in figure [38](#)

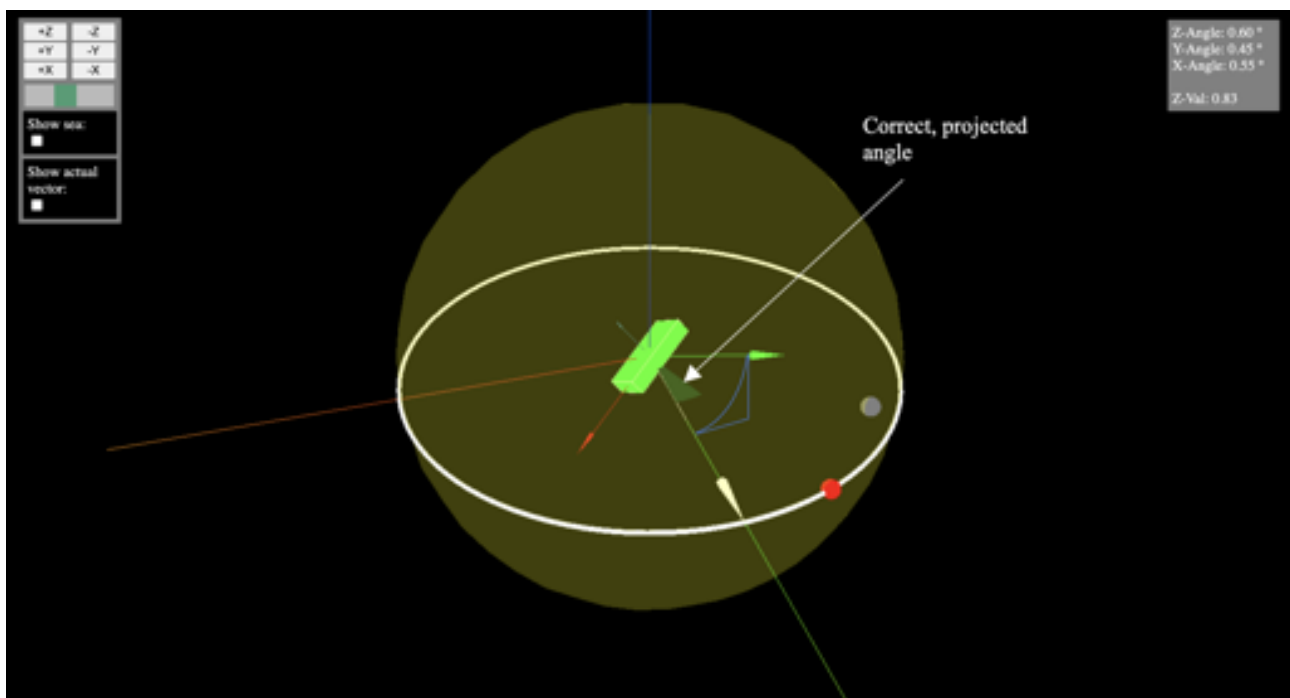


Figure 38: Simulation of the magnetometer with its x, y and z vectors shown. The projected angle is shown.

The simulation in figure [38](#) was made with JavaScript. We built it to demonstrate and understand the tilt-compensation technique and other misunderstandings about the magnetometer. One of the largest misconception which we had was that the heading angle calculated from equation [19](#) was on the calibrated x-y plane. However, the heading angle actually tilts as well with the magnetometer since the axes tilt as well. The tilt compensation technique uses the roll and pitch angles of the body to project the angle back onto the calibrated plane. This is shown in figure [38](#).

7.4.2 Tilt Compensation Equations

Let the following angles be defined [10](#),

γ : roll angle (rotation about the x axis)

ρ : pitch angle (rotation about the y axis)

The roll and pitch angles are that between the y and x axis of the magnetometer, respectively, and the horizontal plane. If the measured magnetic field strengths are viewed as arbitrarily defined points in a Euclidean space, a set of

3D rotation matrices can be applied to 'drag' the arbitrary points back to the calibrated x-y plane.

The rotation matrix for rolling is defined as [10],

$$R_\gamma = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos(\gamma) & \sin(\gamma) \\ 0 & -\sin(\gamma) & \cos(\gamma) \end{bmatrix} \quad (22)$$

The rotation matrix for pitching is defined as [10],

$$R_\rho = \begin{bmatrix} \cos(\rho) & 0 & -\sin(\rho) \\ 0 & 1 & 0 \\ \sin(\rho) & 0 & \cos(\rho) \end{bmatrix} \quad (23)$$

The measurements of the magnetic field at a tilted state can be multiplied by these rotation matrices to get the theoretical measurements at the calibrated x-y plane. Isolating the rotated magnetic field strengths, x' and y' , gives,

$$x' = x\cos(\rho) + z\sin(\rho) \quad (24)$$

$$y' = x\sin(\gamma)\cos(\rho) + y\cos(\gamma) - z\sin(\gamma)\cos(\rho) \quad (25)$$

Then, the new tilt compensated heading can be calculated as,

$$\text{Compensated Heading, } \phi_C = \arctan\left(\frac{y'}{x'}\right) \quad (26)$$

7.4.3 Getting Roll and Pitch Angles for Tilt Compensation

The values for ρ and γ cannot be obtained from the magnetometer. Instead, another sensor must be employed such as an accelerometer or a gyroscope. Initially, an accelerometer was used. We will see why using just an accelerometer or just a gyroscope brings more problems into play. Instead, a different approach must be taken where a mathematical model combines both these sensors to give a very smooth and realistic reading. In this project, the MPU6050 IMU (inertial measurement unit) was used. It includes an accelerometer, a gyroscope and a temperature sensor. It works with the I2C protocol as well and has the same wiring configurations as the magnetometer (see figure [33]).

Accelerometer Angles:

The accelerometer measures acceleration in the x, y and z directions. At rest and level, the x and y readings are approximately 0 and the z direction reads approximately -9.8 as this is the gravitational acceleration. If x_a, y_a and z_a are accelerometer readings, the pitch and roll angles are then,

$$\gamma_a = \arctan\left(\frac{y_a}{z_a}\right) \quad (27)$$

$$\rho_a = \arctan\left(\frac{x_a}{z_a}\right) \quad (28)$$

The major issue with the accelerometer angles is that it also reads the unwanted sudden jerks that the boat experiences due to rotation or waves.

Gyroscope Angles:

The gyroscope measures angular velocity about the x, y and z directions. At rest, these readings are approximately zero. The angles can be obtained by continuously integrating the angular velocities. If the initial roll and pitch angles

are, γ_0 and ρ_0 , we have,

$$\gamma_g = \gamma_0 + \int_0^t \omega_\gamma(t) dt$$

$$\rho_g = \rho_0 + \int_0^t \omega_\rho(t) dt$$

where ω_γ and ω_ρ are the angular velocities. The Arduino will, however, sample values (at a high frequency). Writing the above equations in numerical integration form gives,

$$\gamma_{g,k+1} = \gamma_0 + (\omega_{\gamma,k} * \Delta t) \quad (29)$$

$$\rho_{g,k+1} = \rho_0 + (\omega_{\rho,k} * \Delta t) \quad (30)$$

where k is the iteration number and Δt is the time difference since the last iteration.

In order to keep these values accurate, a timer/counter must be programmed which takes the same time to reach the TOP/MAX value each iteration. The following code sets up a timer with a 1 ms delay. If the counter variable exceeds `timeForOneIteration` (which is 20 ms), then the program will update the gyroscope angles by equations [29](#) and [30](#).

```
...
int timeForOneIteration = 20; // ms
volatile int count = 0;

int main(void){
    if(counter >= timeForOneIteration){ // in milliseconds
        Calculate_Acc_Gyr_Kalman_Angles();
    }
    ...
}

void Calculate_Acc_Gyr_Kalman_Angles(){
    ...
    // Integrate gyro rate
    xAngleGyr += (timeForOneIteration * 0.001) * Xg;
    yAngleGyr += (timeForOneIteration * 0.001) * Yg;
    ...
    counter = 0;
}

ISR(TIMER0_COMPA_vect) {
    counter++;
}

void Timer0_init(){
    TCCR0A |= (1 << WGM01);
    OCR0A = 0xF9; // equivalent of delay which is 1 millisecond (using formula)
    TCCR0B |= (1 << CS01) | (1 << CS00); // 64 prescaler
    TCNT0 = 0;
    TIMSK0 |= (1 << OCIE0A);
}
```

A disadvantage in using a gyroscope is that even when there is no physical rotation, the gyroscope still reads a small value which builds up the angle overtime. This is known as *gyrodrift*. More precise numerical integration can be executed such as Trapezoidal or Simpsons integration. However, the drift, although smaller now, will still exist and grow.

7.4.4 Sensor Fusion with Kalman Filtering

Kalman Filters are popular in combining sensor data in the presence of noise. Essentially, they predict the state of a system based on previous state(s). A diagram of the sensor fusion with the x axis (roll angle) is shown in figure 39

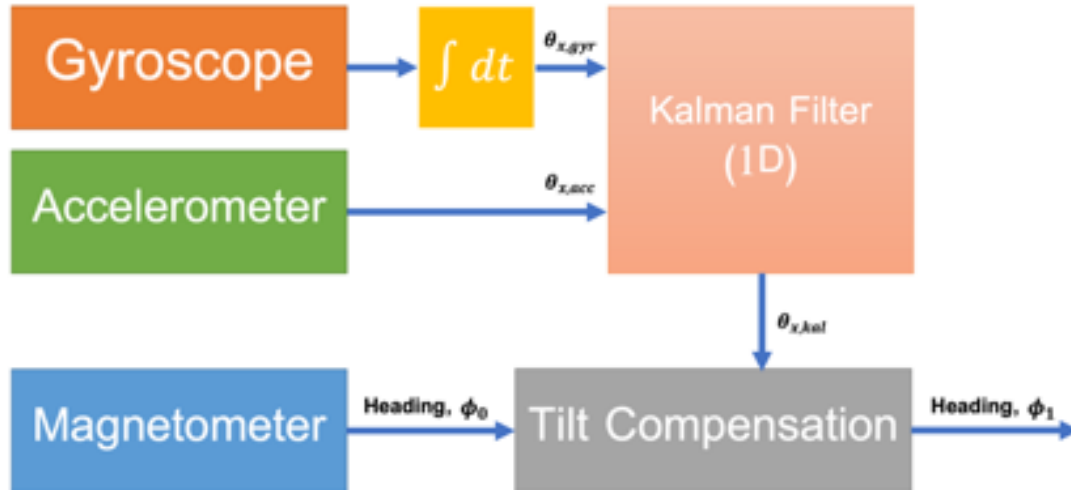


Figure 39: Diagram of the Tilt Compensation algorithm with Sensor Fusion and Kalman Filtering for the roll angle. The pitch angle goes through the same process and then to the Tilt Compensation block.

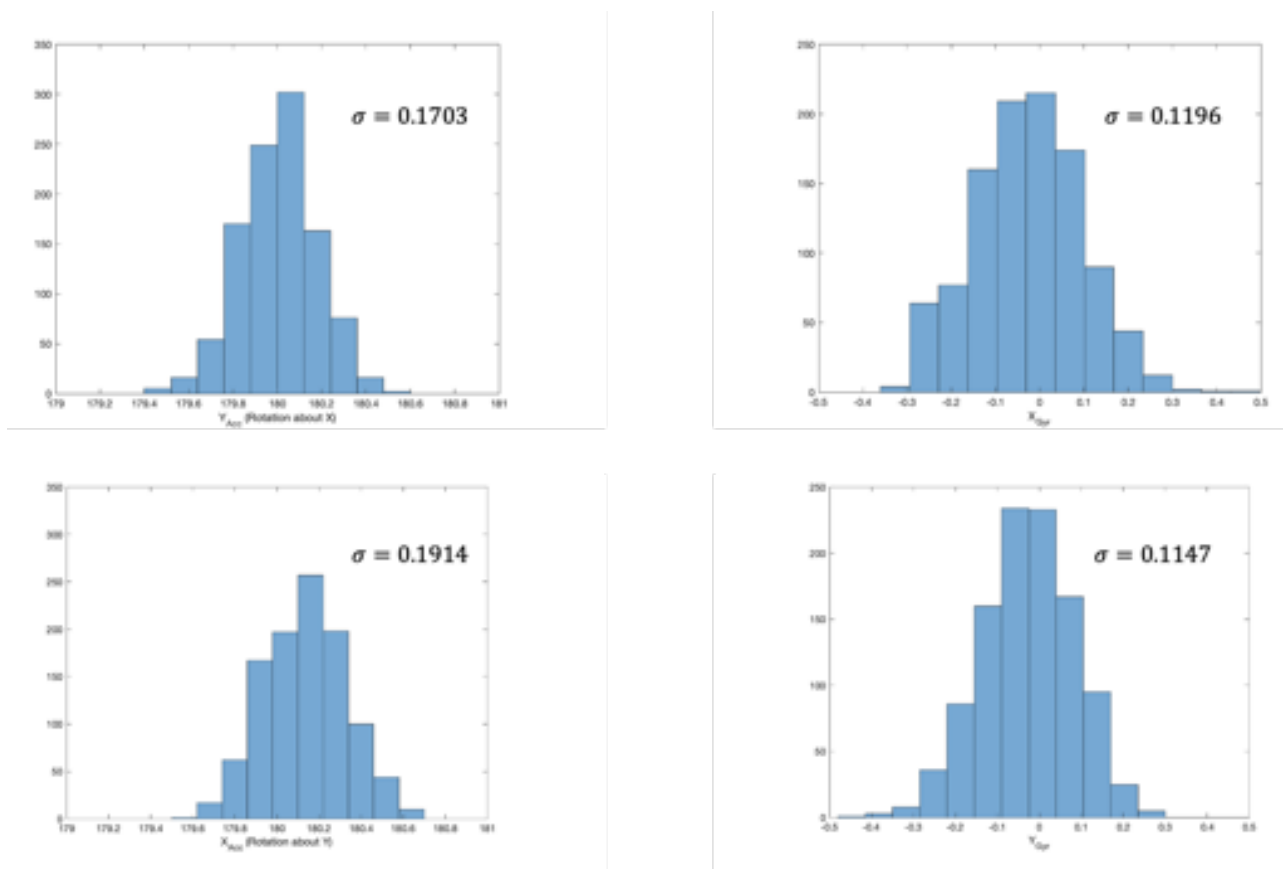


Figure 40: Histogram of the readings from the accelerometer and gyroscope with the standard deviation shown. Data and plots were processed/produced in MatLab

To build the Kalman Filter, the standard deviation, σ , of the gyroscope and accelerometer readings must be deter-

mined. This is done by keeping the sensors steady and recording the values over a time. The variation in the values will result in a normal distribution. According to figure 40, the standard deviation of the accelerometer and gyroscope is on average ≈ 0.18 and ≈ 0.117 respectively.

The following function calculates the Kalman Filter angles and uncertainties 11:

```
float KalmanAnglePrediction = 0;
float KalmanUncertaintyPrediction = 0;
void Calculate_Kalman_Filter(float KalmanState, float KalmanUncertainty, float KalmanInput,
    float KalmanMeasurement, int milliseconds){
    float standardDeviationGyro = 0.12;
    float standardDeviationAcc = 0.18;
    float deltaTime = milliseconds * 0.001;

    KalmanState += deltaTime * KalmanInput;
    KalmanUncertainty += deltaTime * deltaTime * (standardDeviationGyro *
        standardDeviationGyro);

    float KalmanGain = KalmanUncertainty * 1 / (1 * KalmanUncertainty + (standardDeviationAcc
        * standardDeviationAcc));
    KalmanState += KalmanGain * (KalmanMeasurement - KalmanState);
    KalmanUncertainty *= (1 - KalmanGain);

    KalmanAnglePrediction = KalmanState;
    KalmanUncertaintyPrediction = KalmanUncertainty;
}
```

The Calculate_Kalman_Filter function is a one-dimensional Kalman filter for sensor fusion. It incorporates gyroscope and accelerometer data to update the Kalman state and uncertainty, refining the estimation of a system's state. The function utilizes time information and predefined standard deviations for gyro and accelerometer measurements to enhance the accuracy of the Kalman filter predictions.

7.4.5 Tilt Compensation Results

The following function uses the calculated Kalman angles for roll and pitch and equation 26 to return a compensated heading value. Notice that the calibration offsets have also been added to the magnetometer readings for x and y.

```
float getCompensatedHeading(){
    int x = Magneto_GetX() + offsetX;
    int y = Magneto_GetY() + offsetY;
    int z = Magneto_GetZ();

    float xAccAngle = KalmanAngleX / RAD_TO_DEG;
    float yAccAngle = -1 * KalmanAngleY / RAD_TO_DEG;

    float xComp = x*cos(yAccAngle) + y*sin(xAccAngle)*sin(yAccAngle)-z*cos(xAccAngle)*sin(
        yAccAngle);
    float yComp = y*cos(xAccAngle) - z*sin(xAccAngle);

    float compHeading = Magneto_calculateHeading(xComp, yComp);
    return compHeading;
}
```

We have seen that the x and y axes have a very predominant hard iron offset. The only drawback of this implementation is that there is no calibration (hard or soft) for the z axis.

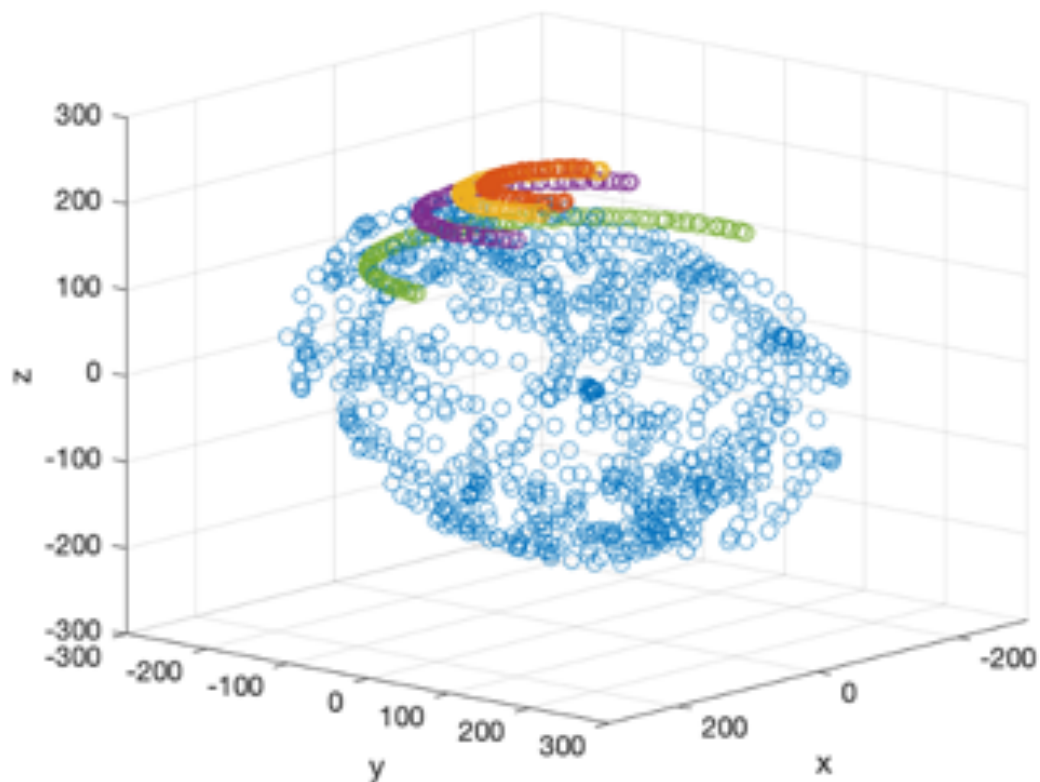


Figure 41: X-Y Calibrated 3D plot of x, y and z readings from the magnetometer.

According to figure 41, the z axis is quite stretched horizontally. This signifies a soft-iron offset which makes sense as there are many wires below the magnetometer. Moreover, there is no hard-iron calibration either. Because the z-axis was never calibrated, there still might be errors/incomplete tilt compensation.

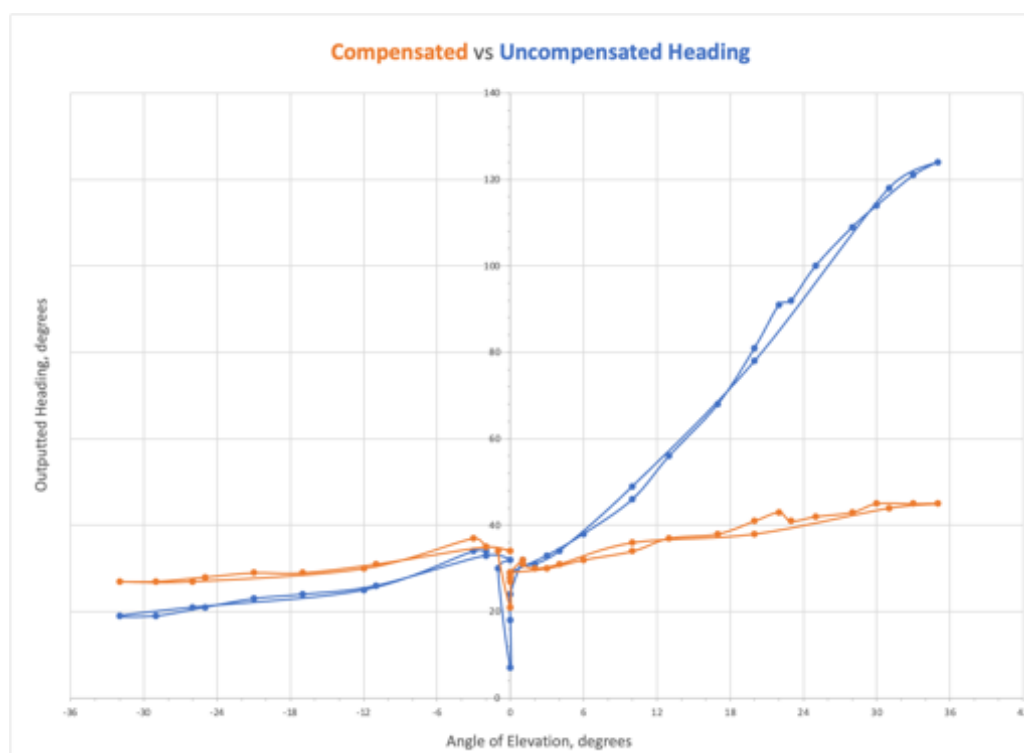


Figure 42: Comparison of compensated heading and heading for an elevation of up to 36°. Uncompensated - blue, compensated - orange.

According to figure 42, for an elevation of approximately 35° , the Tilt Compensation with the Kalman Filter results in an unwanted variation of only 15 degrees whereas the uncompensated method varies by 105° which is a huge improvement.

The sensor fusion with Kalman Filter seems to produce very accurate results as shown by figures 43 and 44. In comparison with gyroscopic integration, it is much less prone to very rapid vibrations. For instance, referring to figure 43, point A introduces many vibrations. The gyroscopic angle is seen to drift off due to limited processing frequency and noise. Towards the end, there is a clear shift in the gyroscopic integration.

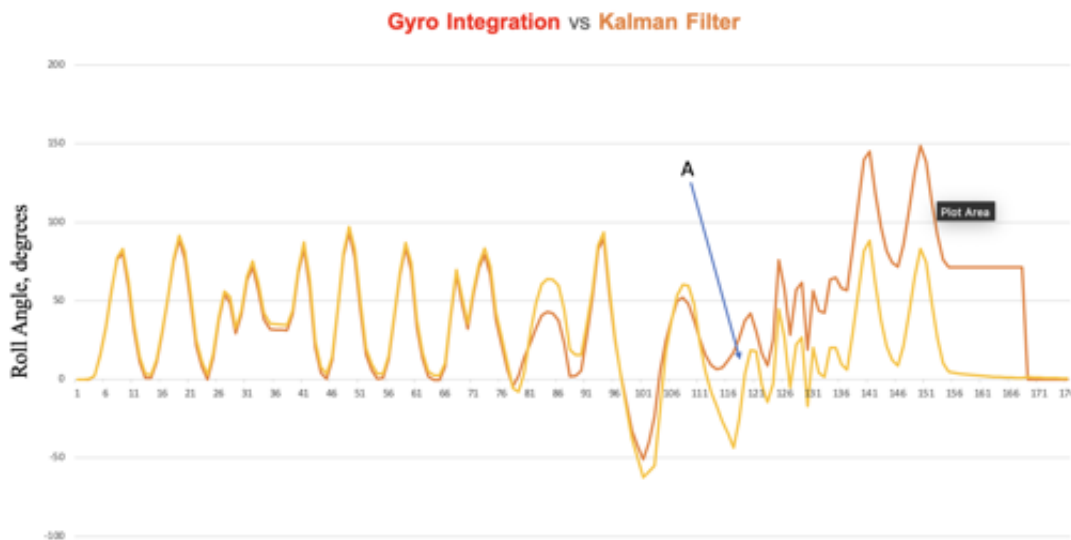


Figure 43: Roll Angle computations with Gyroscopic Numerical Integration versus Kalman Filter. At point A, large vibrations cause rapidly evolving variations for the Gyroscopic Integration.

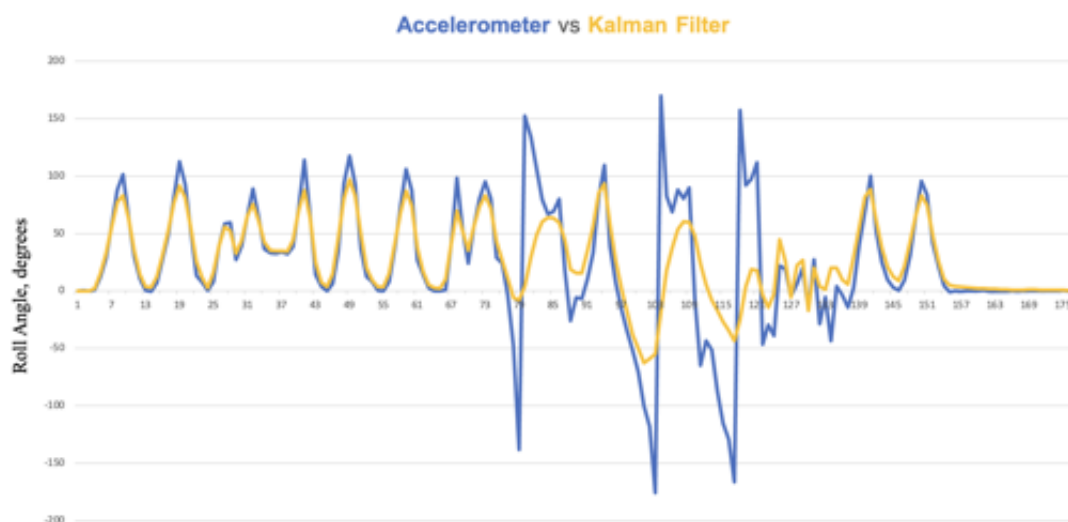


Figure 44: Roll Angle computations with Accelerometer Trigonometry versus Kalman Filter.

In comparison with the accelerometer trigonometry, the Kalman Filter avoids the huge variations. In a steady state or gradual movement however, the accelerometer agrees well with the Kalman Filter. This makes sense since the additional acceleration is negligible.

7.5 Navigation

7.5.1 Motor Control, Navigation Setup and Algorithm

The T200 thrusters require a PWM signal to run. The following definitions correspond to the max thrusts and stop.

```
#define MAX_FORWARD 1900 // pwm max cw thrust (1900us)
#define MAX_REVERSE 1100 //pwm max ccw (1100us)
#define STOP 1500 // pwm stop(1500us)
```

Then, after initializing Timer4 for PWM at 200Hz and setting pins D6 and D7 as output, a simple function is used to set the thrust for each motor.

```
void Set_Thrust(int pulse_width, int pulse_width2) {
    OCR4A = pulse_width;
    OCR4B = pulse_width2;
}
```

The navigation setups are shown in the figures [45](#) and [46](#). A proportional controller was used to navigate the boat. The idea behind this is that the further the boat faces away from the destination heading, the faster the boat will turn to correct its direction. This relationship will be proportional.

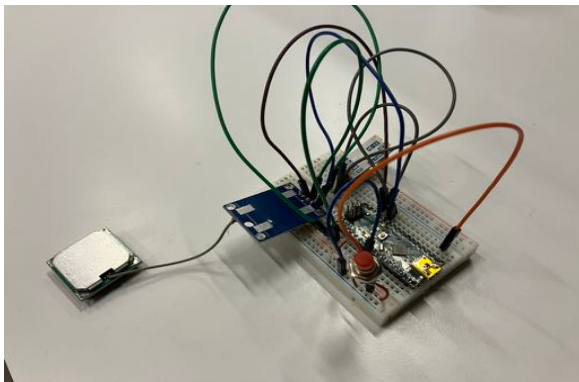


Figure 45: Bracelet test setup on breadboard with GPS, RF, Button and Arduino Nano.

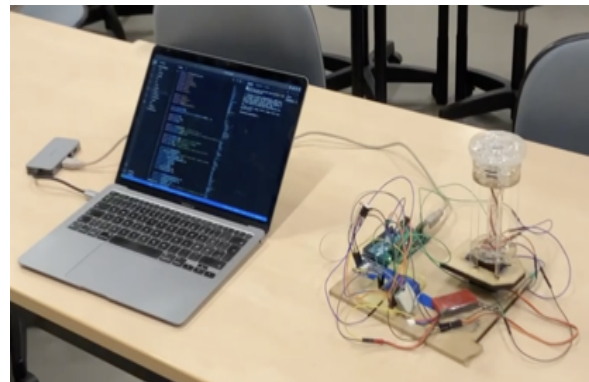


Figure 46: Navigation setup on a board with GPS, RF, Magnetometer, MPU6050 and Arduino Mega.

The following process illustrates the navigation algorithm:

1. Get heading (compensated)
2. Get destination heading
3. Calculate the angle and direction to rotate
4. Calculate the new motor speeds based on the proportional controller
5. Set the motor speeds
6. Calculate the distance to the destination
7. If the distance is less than 5 m, the target has been reached
8. Repeat

7.5.2 Destination Heading

The destination heading is the angle from north, about the boat and to the destination (the bracelet). This will indirectly instruct the boat the angle and direction to turn.

Since the curvature of the Earth between the boat and the destination is negligible, it can be assumed that the following ratios relation holds true,

$$\frac{\text{Latitude}_{\text{dest}} - \text{Latitude}}{\text{Longitude}_{\text{dest}} - \text{Longitude}} \propto \frac{Y_{\text{dest}} - Y}{X_{\text{dest}} - X}$$

Here, Y_{dest} and X_{dest} are the vertical and horizontal positions on the water of the destination. Hence, the destination heading can be deduced from just the latitudes and longitudes:

$$\theta = \arctan\left(\frac{\Delta_{lat}}{\Delta_{long}}\right) \quad (31)$$

where $\Delta_{lat} = (\text{Latitude}_{\text{dest}} - \text{Latitude})$ and $\Delta_{long} = (\text{Longitude}_{\text{dest}} - \text{Longitude})$. Equation 31 only gives the angle to the horizontal axis. To get the full heading from North, the following transformations are applied (shown in code):

```
float deltaLat = currentDestLatitude - currentLatitude; // y (kind of)
float deltaLong = currentDestLongitude - currentLongitude; // x (kind of)
float ratio = ((float)deltaLat) / ((float)deltaLong);
float angle = fabs(atan(ratio) * 180.0 / ((float)PI)); // absolute value taken
float destHeading = 0;
if(deltaLong <= 0 && deltaLat >= 0){ // transformations
    destHeading = 90.0 - angle;
} else if(deltaLong <= 0 && deltaLat <= 0){
    destHeading = 90.0 + angle;
} else if(deltaLong >= 0 && deltaLat <= 0){
    destHeading = 270.0 - angle;
} else if(deltaLong >= 0 && deltaLat >= 0){
    destHeading = 270.0 + angle;
}
}
```

7.5.3 Angle and Direction to Rotate

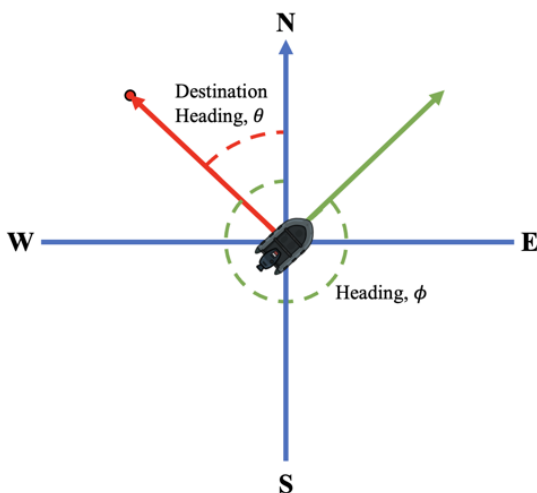


Figure 47: Case 1: When $\theta < \phi$

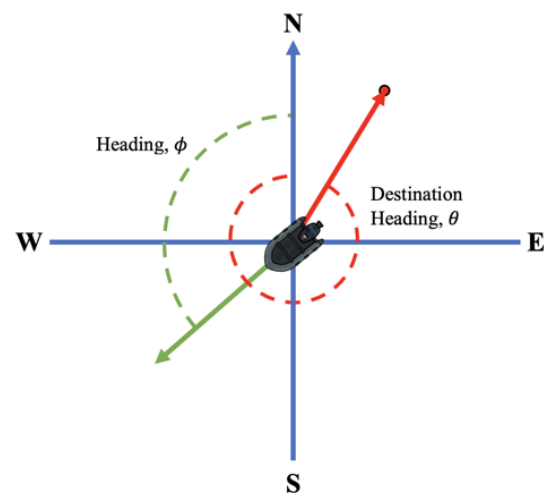


Figure 48: Case 2: When $\theta > \phi$

The two cases in figures 47 and 48 are a) when the destination heading is less than the boat's heading and b) when the destination heading is larger than the boat's heading. What needs to be determined now, is the obtuse or acute angle in between the heading vector and the destination heading.

Describing how the angles are obtained is difficult but easily understandable in if statements as in the code below.

```
float destHeadMinusHead = destHeading - heading;
float headMinusDestHead = heading - destHeading;

if(destHeadMinusHead >= 0){ // CASE 1
    if(destHeadMinusHead >= 180){ // turn right
        *turnRightOrLeft = 0;
        *deltaAngle = (360 - destHeadMinusHead);
    } else { // turn left
        *turnRightOrLeft = 1;
        *deltaAngle = destHeadMinusHead;
    }
} else { // CASE 2
    if(headMinusDestHead >= 180){ // turn left
        *turnRightOrLeft = 1;
        *deltaAngle = (360 - headMinusDestHead);
    } else { // turn right
        *turnRightOrLeft = 0;
        *deltaAngle = headMinusDestHead;
    }
}
```

Here, deltaAngle and turnRightOrLeft variables to show the magnitude of the angle between the boat heading and the destination heading and which direction to turn respectively.

7.5.4 Applying a Proportional Controller

The turning effect of the boat will be dependent on the deltaAngle calculated from the code above. When turning, one motor at full thrust and the other slightly lower to turn the boat.

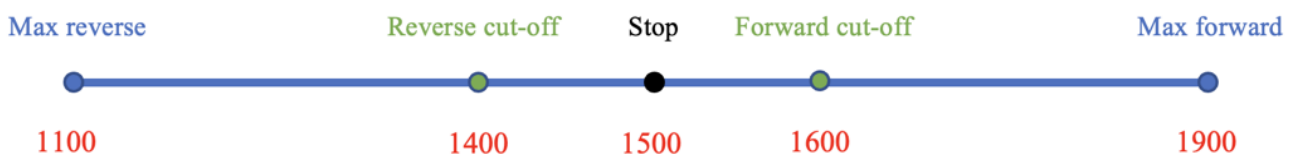


Figure 49: Diagram of Proportional Controller with PWM.

Figure 49 shows the PWM control for the motors. The idea is that if the boat is 180° from the destination heading, one of its motors should go at full thrust whereas the other should go at the cut-off. In this way, the boat drives forward while also turning. As the boat's heading approaches the destination heading, the thrust in the latter motor increases closer to Max forward or Max reverse proportional. This change is proportional to deltaAngle. This concept can be expressed as a single equation:

$$\frac{\text{deltaPWM}}{\text{Max forward} - \text{Cutoff}} = \frac{\text{deltaAngle}}{180} \quad (32)$$

where deltaPWM is the reduction in PWM for the motor from Max forward. Solving for deltaPWM gives (in code),

```
float deltaPWM = (deltaAngle / 180.0) * (MAX_FORWARD - CUTOFF_FORWARD);
```

Then, using the Set_Thrust function, a new thrust can be set based on deltaPWM.

7.5.5 Calculating Distance and Checking Arrival

The distance between the boat and the destination needs to be calculated to check if the boat has arrived at the destination. The difference in latitude and longitude between the boat and the destination correspond to a certain horizontal, d_h , and vertical, d_v , distance. These distances are,

$$d_v = \frac{\text{Latitude}_{\text{dest}} - \text{Latitude}}{360} * 2\pi R$$

$$d_h = \frac{\text{Longitude}_{\text{dest}} - \text{Longitude}}{360} * 2\pi R$$

Then, the distance is,

$$d = \sqrt{d_v^2 + d_h^2} \quad (33)$$

The following implements this equation in code.

```
// Find distance to target
float y = (destLatitude - latitude) * (2 * PI * 6371) / 360.0;
float x = (destLongitude - longitude) * (2 * PI * 6371) / 360.0;
dist = sqrt(pow(x, 2) + pow(y, 2)) * 1000.0;
```

When this distance goes below a certain value (in this project, it was 5 m), the boat will stop and wait until the button on the boat is clicked. This means the person has held onto the boat. At this point, the destination is now set to the boat's initial coordinates and the program repeats. This process is described below.

```
if(dist <= 5.0){
    if(goingBackAfterPickUp == 0){ // going to pickup, reached person
        // Implement button press, motor stop wait
        goingBackAfterPickUp = 1;
    } else if(goingBackAfterPickUp == 1){ // going back, reached shore
        // Implement stop motors and everything
        goingBackAfterPickUp = 0;
        break; // ends the whole program
    }
}
```

7.5.6 Testing the Navigation System

The navigation system was tested by imagining a person walking as the boat. The person held the navigation system and walked with it while viewing the serial monitor from the computer. Commands such as 'Turn Right/Left' and 'Reached Destination' were printed to the serial monitor. Examples of prints are shown below.

First log at initial position:

```
Lat: 54.912415    Long: 9.779622
DestLat: 54.912254    DestLong: 9.778997
heading: 109, destHead: 104
KalmanX: 0.077, KalmanY: -0.160
Distance: 71.7
Turn right
```

Arrived at bracelet location:

```
Lat: 54.912251    Long: 9.779040
DestLat: 54.912254    DestLong: 9.778997
heading: 150, destHead: 85
KalmanX: 2.581, KalmanY: 2.263
Distance: 4.8
Turn right
Going back now...
```

Reached back at initial position:

```
Lat: 54.912395      Long: 9.779579
DestLat: 54.912418    DestLong: 9.779615
heading: 31, destHead: 302
KalmanX: 3.486, KalmanY: 2.588
Distance: 4.8
Turn right
Finished rescue mission...
Exiting program...
```

Logging important values such as heading and Distance was very useful in debugging errors. Figure 50 shows the final test done outside. Each latitude and longitude coordinate on the serial monitor's logs were extracted and put on a map. The navigation system took us exactly to the destination (in front of Sønderborg St.) and back (last blue coordinate on the right).

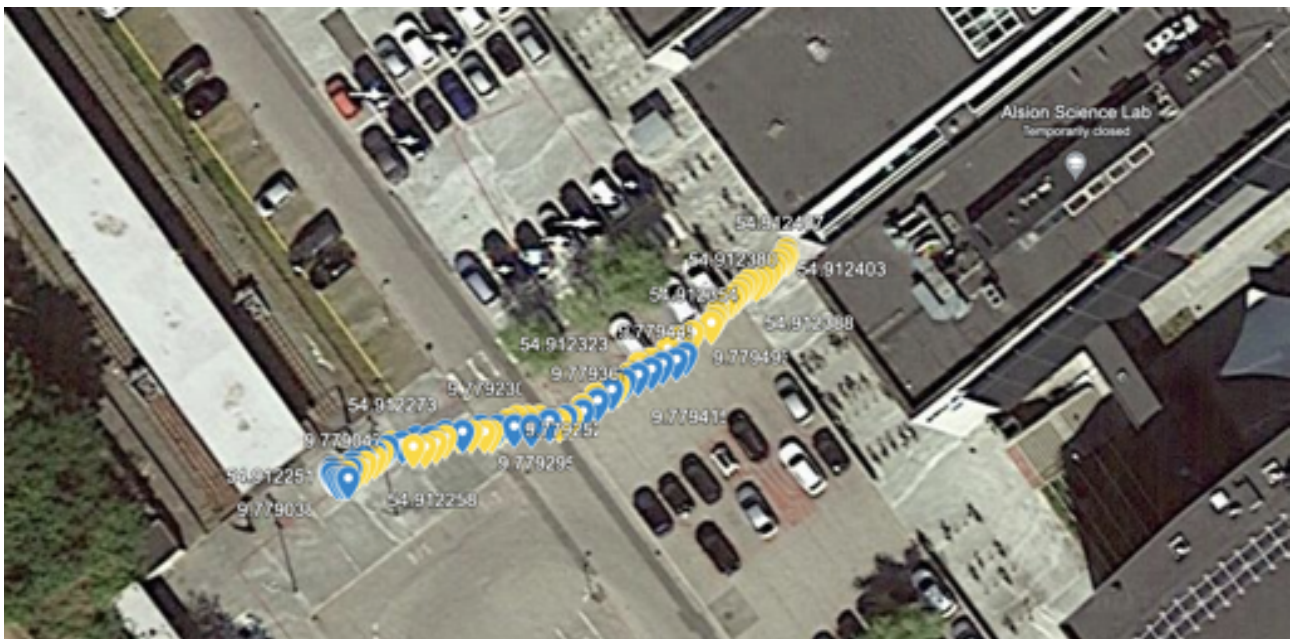


Figure 50: A birds-eye view of the final navigation test done outside of the campus. The yellow points represent on the way to the bracelet and the blue points represent the way back.

8 Limitations and Conclusion

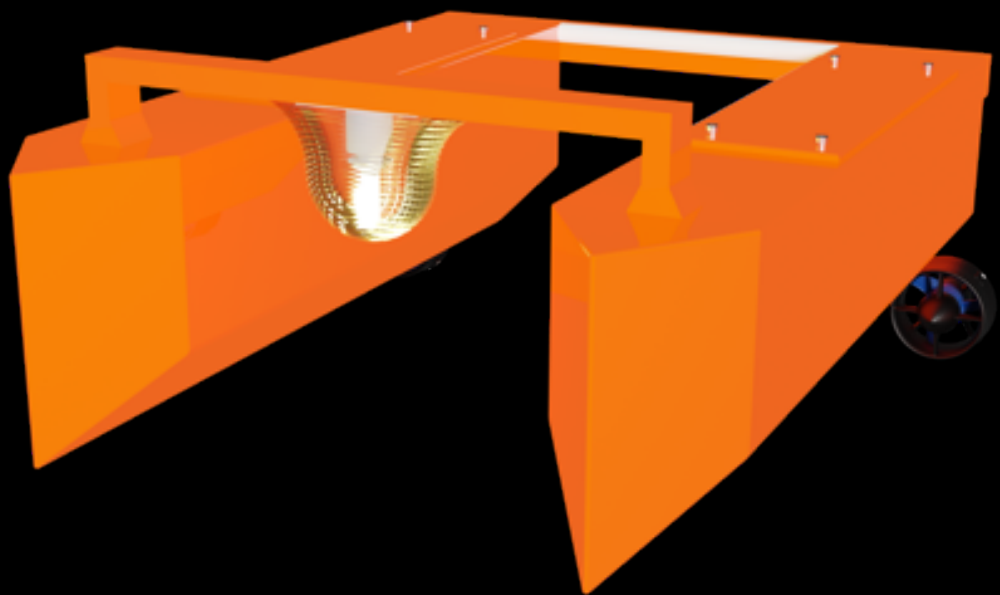
1. Limitations

From the latest testing results, each individual components; mechanical, electrical or software, are recorded to meet the mandatory product requirements and functionalities. But due to time constraints, the boat itself has only been confirmed to work under laboratory settings. In addition, the analog sensor aspect of the project has been hindered by sensor equipment that despite the best testing efforts, seem to be faulty. Because of this, the circuit and software setup of this component are confirmed to function in simulations and not successfully tested.

2. Conclusion

Semester Project 3 is an exercise to investigate, understand, choose, and implement suitable electronic technologies towards a prototype that matches the user requirements set forth in the beginning of this report. The project is divided into three departments to conform to ISO 14971's need of clear assignment of responsibilities. Major risks identified by the project group were assessed and eventually reduced within acceptable limits, in simulation or component testing. Following the problem formulation, the requirements are further broken down into individual design decisions and technical specifications to be fulfilled. These specifications are not only aimed at fulfilling user requirements, but also at minimizing risks of failure as far as practicable. The project demonstrates a working rescue boat system prototype that is experimentally functional in its anonymous work cycle according to the user and project requirements set forth in the beginning of this report. Further improvements may include finding a more suitable analog sonar setup, further optimizing the mechanical assembly to absorb shocks to the system, more detailed risks management protocols and integrating all subsystems for real-world testing.

Appendix

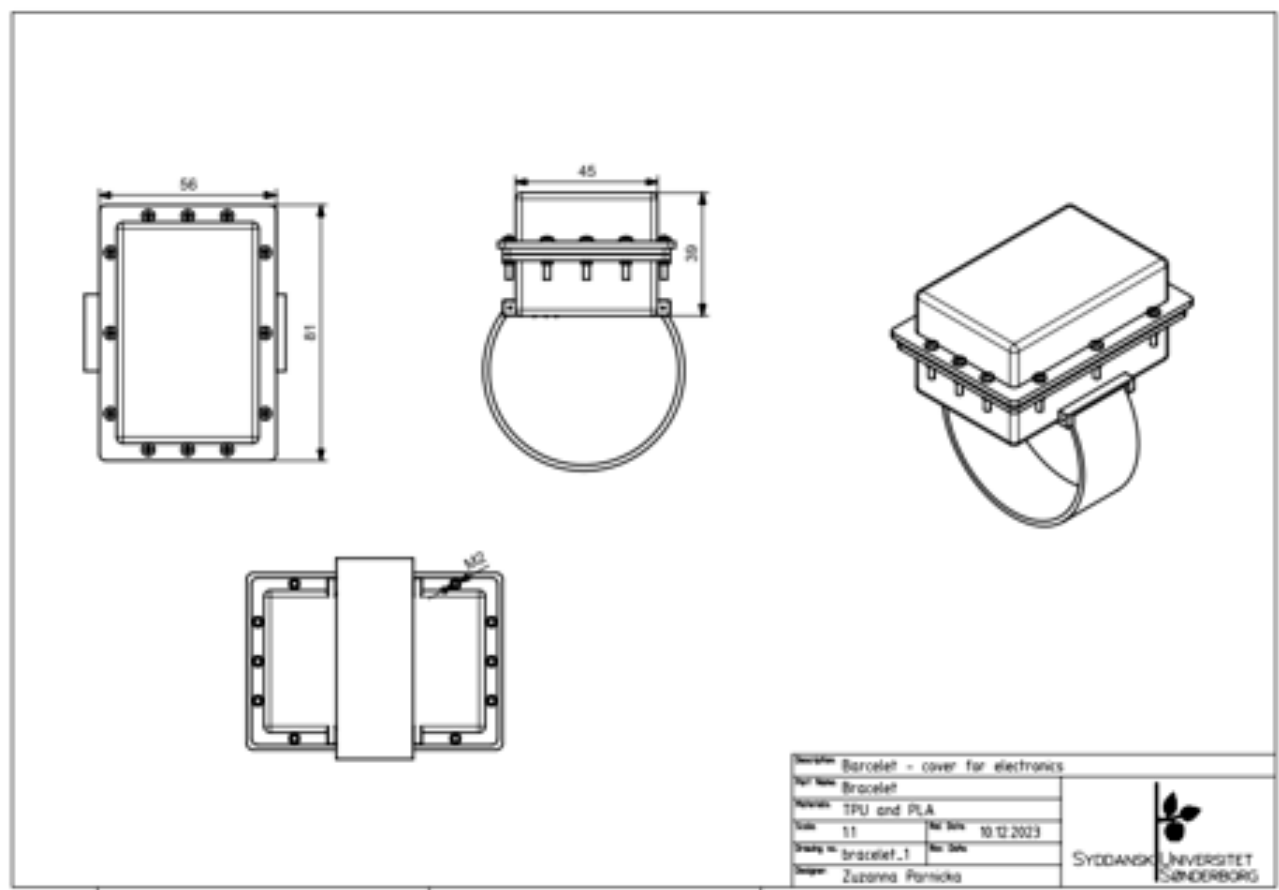


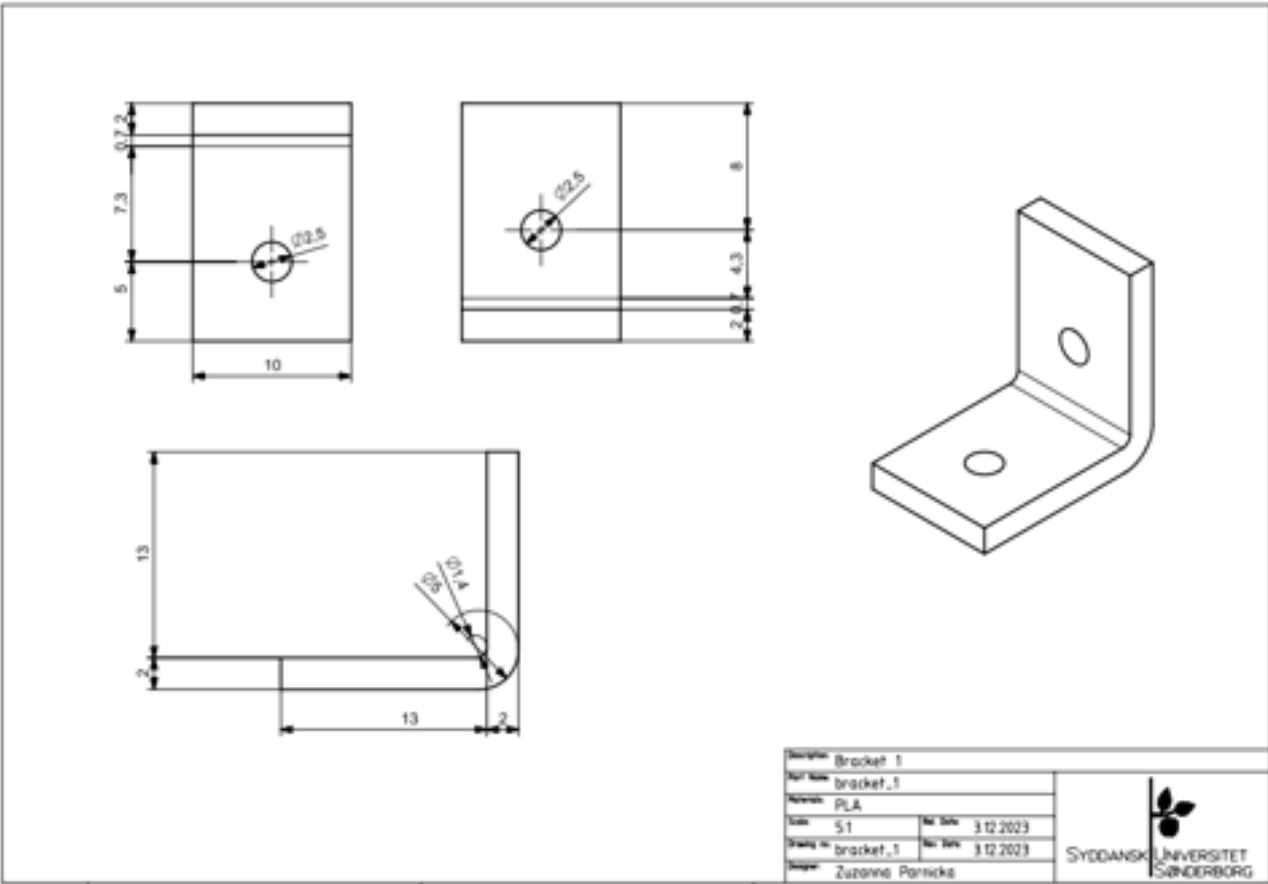
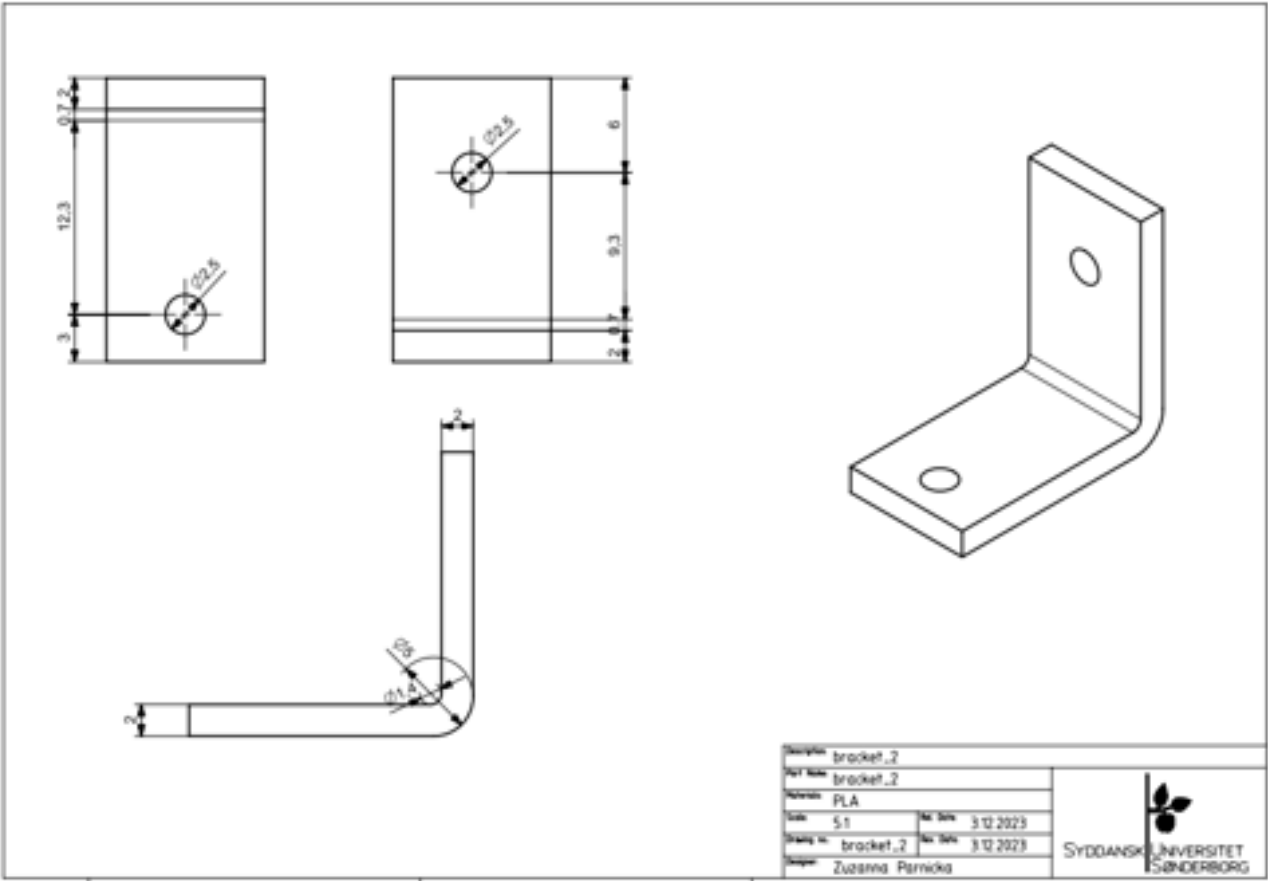
A.1 Survey

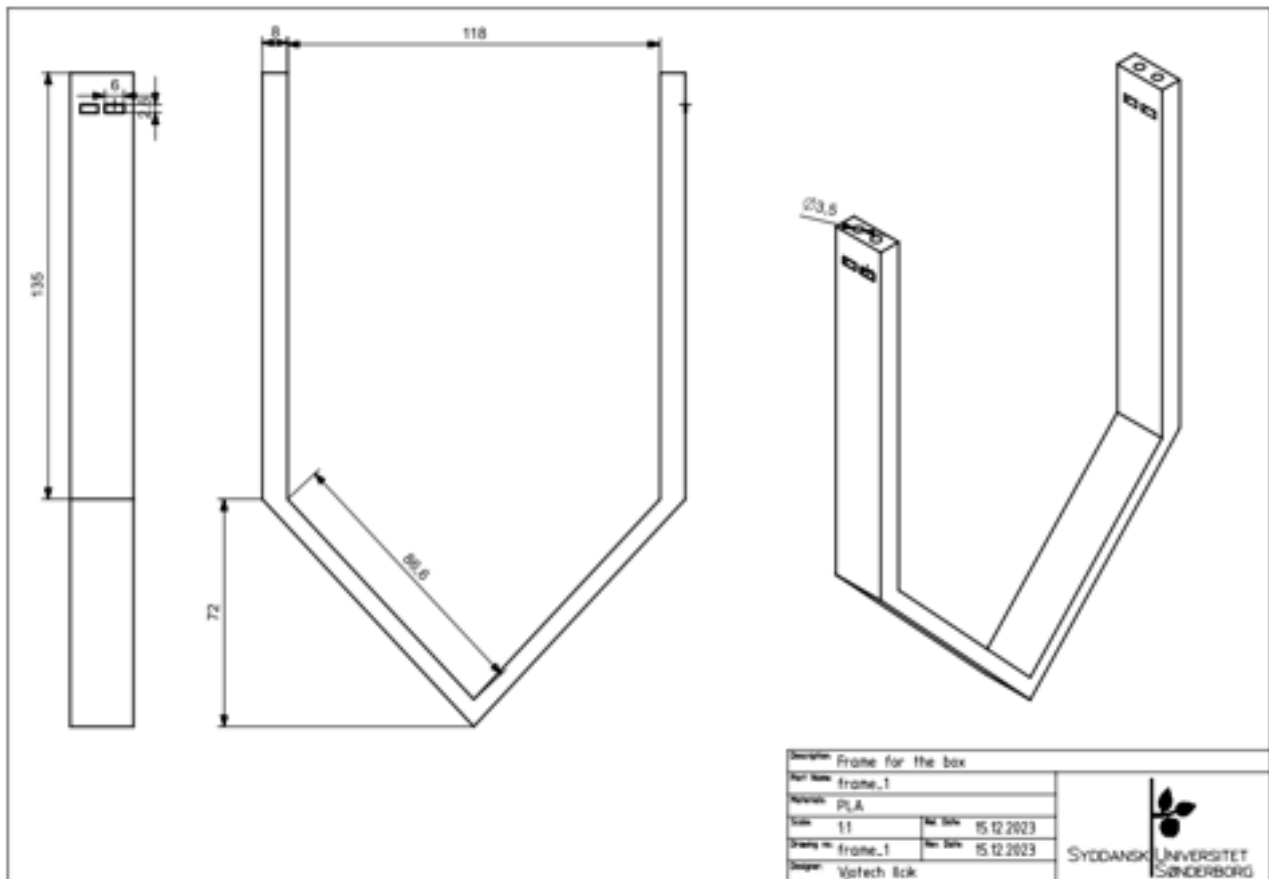
1. The autonomous assistance boat can be a helpful tool for saving people overboard.
Yes (100%)
No (0%)
2. The autonomous assistance boat can be a helpful tool for a lifeguard.
Yes (100%)
No (0%)
3. Shape: (single choice)
 - (a) The person should be able to grab on it, similar to safety ring. (20%)
 - (b) The person should be able to grab on it and lay on it, like a stretcher. (80%)
 - (c) There should be a platform on a boat, that allows the lifeguard to start saving the sufferer after getting to them on the water. (20%)
4. Accessories: (multiple choice)
 - (a) Camera - document the mission (20%)
 - (b) Warning lights (100%)
 - (c) First aid kit (40%)
 - (d) Warning sound (80%)
 - (e) Showing temperature of water (0%)
 - (f) 5-min oxygen bottle (40%)
5. Is it important that the boat/buoy is helping the lifeguard to get to the person?
Yes (100%)
No (0%)
6. Does it make sense that there will be something that holds the lifeguard to the boat so he can take care of the drowning person during coming back to the shore?
Yes (100%)
No (0%)
7. Does the colour of the boat matter?
Yes (60%)
No (40%)
8. If the colour matters, what colour should it be? (Specify if required by law)
Written answer by interviewees:
 - (a) Bright, regulated by rules for country's regulations.
 - (b) Bright, visible, red would be the best.
 - (c) Bright, fluorescent.

9. From your experience what is the average distance [m] from shore to the victim? (multiple choice)
- (a) 100 (75%)
 - (b) 200 (25%)
 - (c) 300 (0%)
 - (d) 400 (0%)
 - (e) 500 (0%)
 - (f) Other: (0%)
10. What is the maximum time [min] in which the lifeguard should get to the drowning person so the sufferer can be saved in order to survive? (multiple choice)
- (a) 1 (0%)
 - (b) 2 (20%)
 - (c) 3 (20%)
 - (d) 4 (20%)
 - (e) 5 (20%)
 - (f) 6
 - (g) More than 6 (20%)
11. What is the biggest distance [m] from the shore that the sufferer is saved at?
Written answer by interviewees:
- (a) It depends on the time of apnoea, not the distance.
 - (b) 90.
12. What was the longest time [min] it took you to complete a rescue (time spent in water)? (multiple choice)
- (a) Up to 10 (40%)
 - (b) Up to 20 (40%)
 - (c) Up to 40 (20%)
13. Other suggestions and features that you would like to have on your autonomous assistant.
No given answers by interviewees.

A.2 Technical Drawings







A.3 Waterproofing research

1. Electronic Epoxy Adhesive Glue 12

- Does not protect connectors from water.
- Shouldn't be used with high voltage boards.
- Shouldn't be used on pins, switches, buttons that have to be used.
- Components like power transistors, amplifiers etc. may become hot due to being insulated by Epoxy. Which would effect on them being less effective at dissipating heat, shortening the components life span.
- There are different kinds of Epoxy. Some have metal in them, other ones are thermally conductive.
- Epoxy is setting in about 5 min. Therefore, the applying procedure has to be fast.
- It is commonly used with silicon. Mostly silicon is put on mounting points or connectors where Epoxy is likely to crack. Also on the buttons.
- It work process is that it is chemically bonding the two parts together.
- It adhesies to a variety of material, like metals, plastics, ceramics, glass.
- It provides a bond that withstands stress, vibration and shock.
- It is resistant to chemicals, heat, moisture.
- It can be applied to small and large areas
- It has a long curing time, so it might take several hours to fully cure.
- It's not suitable for use on flexible materials.
- The two parts of the adhesive must be mixed in the correct ratio for optimal results.
- It cannot be undone.

2. Silicone¹³

- It can withstand mechanical damage.
- It is messy and hard to use while applying.
- The heat transfer is all right.
- Hides the whole item that is covered by it.

3. Nail polish

- Easy to use.
- Work sufficient with small surfaces.
- Not good to use on elastic surfaces.
- Leaves the visibility of the items.
- Costs about 20 kroner for a bottle.

4. Polyurethane

- Moisture and solvent resistance.
- Class F temp. rating.
- Good dielectric properties.
- Abrasion resistant.
- Flexible.
- Fungicidal.
- It can be done in three ways: dip, spray, brush.
- Single-component urethane coatings are easy to apply; the trade-off, however, is that they have a long cure cycle (up to several days). Two-component urethane coatings have a shorter cure cycle (1-3 days) but are more difficult to apply.

5. Acrylic

- Quick-drying nature.
- Easy to remove.
- Highly resistant to humidity.
- Does not give off a lot of heat while it dries and doesn't shrink as it cures.

6. Para-xylylene

- Or Parylene, a chemical coating done by Chemical Vapor Deposition (CVD) in an atmosphere of Para-xylylene or its derivative. The thin film is generated by the chemical vapor adhering to the part to be coated and polymerized at a threshold temperature of 700C. Within the referenced text, the CVD process is described as followed:
- The part is placed inside a container that allows easy access for possible adjustments and retrievals. Temperature probes need to be placed in more than one location inside the reaction chamber to measure and model the temperature and pressure gradient to ensure homogeneous temperature and pressure distribution along the part. This is done to ensure an even coating of material and the eventual polymerization of the final coating. Polymerization can happen at different temperatures depending on the technique, but the chamber needs to be able to achieve a maximum temperature of 700 degrees. Due to the corrosive properties of the precursor vapor, the chamber needs to be lined with quartz, so that the usual stainless-steel chamber stays intact. The recommended shape for the chamber is a bell-shaped chamber, of relatively small size to ensure isothermal and isobaric internal properties.

- While being extremely interesting and possessing highly robust properties such as anti-abrasion, beyond the waterproofing abilities, the method is too expensive and complex for our current project.

7. Fluoropolymer

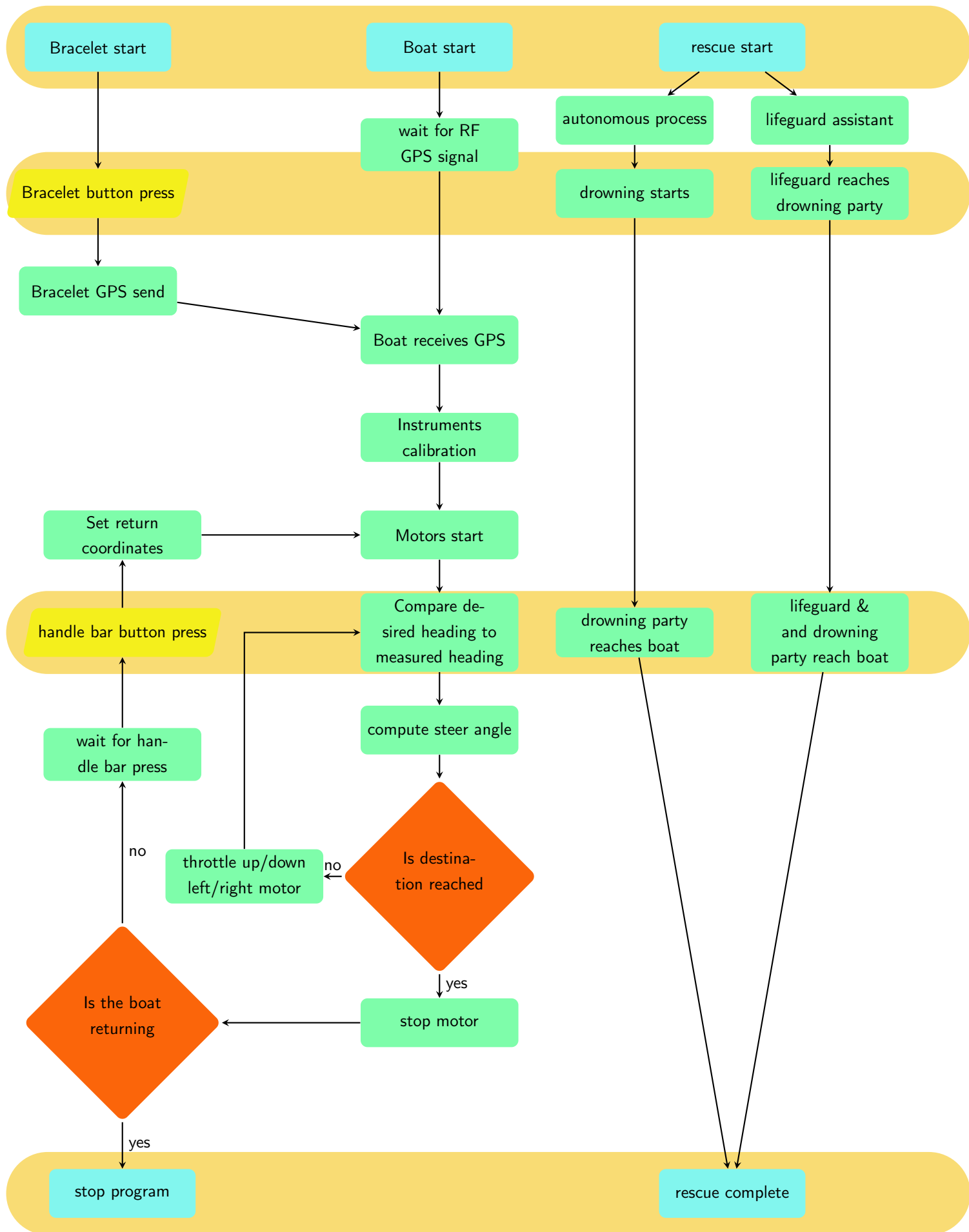
- Similar to the previous method, Fluoropolymers are generally applied as a coating. Some of the more commonly known applications of Fluoropolymers are cookware and clothes, and one of its most known varieties is Teflon. Due to the complexity of application and difficulty acquiring the correct chemical for the project's purposes, the option is eliminated.

8. Waterproof container

- As a secondary waterproofing measure, a waterproof container is an excellent form of waterproofing, being both inexpensive and easy to modify. The setup is easily sealed and unsealed and provide many configurations options.

9. Marine grease

- Marine grease, like other grease, are used for their adhesive properties as well as their ability to lubricate moving machineries. In this application, the adhesive property of grease is needed, but not its ability to lubricate mechanical components, since the only moving components will be the two already lubricated motors. In addition, Marine grease is hard to work with and increases difficulty to modify and repair the project.



Bibliography

- [1] [Online]. Available: https://cdn.sparkfun.com/assets/f/8/d/6/d/ZED-F9P-02B_DataSheet_UBX-21023276.pdf?_gl=1*150sgcj*_ga*MTAxMTI1MDc4OS4xNjgxNzE50TE5*_ga_T369JS7J9N*MTY5Njg3NjM0Mi40LjEuMTY5Njg3NjU1OS42MC4wLjA.
- [2] [Online]. Available: <https://files.banggood.com/2016/11/BN-220%20GPS+Antenna%20datasheet.pdf>.
- [3] [Online]. Available: <https://www.nextpcb.com/blog/waterproof-pcb--nextpcb>.
- [4] [Online]. Available: <https://www.sparkfun.com/datasheets/GPS/NMEA%20Reference%20Manual-Rev2.1-Dec07.pdf>.
- [5] Kosma, *Kosma/minmea: A lightweight gps nmea 0183 parser library in pure c*. [Online]. Available: <https://github.com/kosma/minmea>.
- [6] W. Ewald, *Using the atmega328p standalone • wolles elektronikkiste*, Aug. 2022. [Online]. Available: <https://wolles-elektronikkiste.de/en/using-the-atmega328p-standalone>.
- [7] FastBitLab, *Ascii code - table of ascii characters and example*, Dec. 2022. [Online]. Available: <https://fastbitlab.com/ascii-code/>.
- [8] [Online]. Available: <https://www.electronicwings.com/avr-atmega/magnetometer-hmc5883l-interfacing-with-atmega16>.
- [9] [Online]. Available: <https://www.vectornav.com/resources/inertial-navigation-primer/specifications--and--error-budgets/specs-hsicalibration>.
- [10] A. Kuncar, M. Sysel, and T. Urbanek, *Calibration of triaxial accelerometer and triaxial magnetometer for tilt compensated electronic compass*.
- [11] CarbonAeronautics, *Carbonaeronautics/part-xv-1dkalmanfilter*. [Online]. Available: <https://github.com/CarbonAeronautics/Part-XV-1DKalmanFilter>.
- [12] [Online]. Available: <https://medium.com/@epoxyglue/everything-you-need-to-know-about-electronic-epoxy-adhesive-glue-e550b5e3b517>.
- [13] [Online]. Available: <https://www.thomasnet.com/articles/chemicals/Silicon-Electronics-Casting-Applications/>.